

RTL Design Sherpa

APB PIT 8254 Micro-Architecture Specification 1.0

January 4, 2026

Table of Contents

1 Pit 8254 Mas Index....11

2 APB PIT 8254 - Overview....12

2.1 Overview....12

2.1.1 Figure 1.1: APB PIT 8254 Block Diagram....12

2.2 Functional Description....13

2.2.1 Key Features....13

2.2.2 Applications....13

2.2.3 Design Philosophy....14

2.3 Waveforms....14

2.3.1 Waveform 1.1: Mode 0 Terminal Count....14

2.3.2 Waveform 1.2: Mode 2 Rate Generator (reference only - not implemented)....14

2.3.3 Waveform 1.3: Mode 3 Square Wave Generator (reference only - not implemented)....15

2.3.4 Waveform 1.4: Gate Control....15

2.3.5 Waveform 1.5: Readback Command (reference only - not implemented)....16

2.4 Design Notes....16

2.4.1 Comparison with 8254....16

2.4.2 Design Scope....17

3 APB PIT 8254 - Architecture....18

3.1 Overview....18

3.1.1 Figure 1.2: APB PIT 8254 High-Level Block Diagram....18

3.1.2 Module Hierarchy....18
3.1.3 Three-Layer Architecture....19
3.2 Functional Description....19
3.2.1 Data Flow....19
3.2.2 Figure 1.3: APB PIT 8254 Register Write and Read Paths....20
3.2.3 Counter Control State....20
3.2.4 Control Flow....21
3.3 Timing....21
3.3.1 Clock Domains....21
3.3.2 Figure 1.4: APB PIT 8254 Clock Configuration....22
3.4 Design Notes....22
3.4.1 Reset Behavior....22
4 APB PIT 8254 - Clocks and Reset....24
4.1 Overview....24
4.1.1 Figure 1.5: APB PIT 8254 Clock Configuration....24
4.2 Ports....24
4.2.1 Clock Signals....24
4.3 Functional Description....25
4.3.1 Clock Enable Signal....25
4.3.2 Reset Signal....25
4.4 Timing....27
4.4.1 Clock Domain Crossing (CDC)....27
4.4.2 Multi-Clock Timing Constraints....28
4.5 Design Notes....28

4.5.1 Clock Gating....28
5 APB PIT 8254 - Acronyms and Terminology....30
5.1 References....30
5.1.1 Acronyms....30
5.1.2 Terminology....31
5.1.3 Register Field Access Types....32
5.1.4 SystemRDL Concepts....33
5.1.5 Design Architecture Terms....33
6 APB PIT 8254 - References....34
6.1 References....34
6.1.1 Primary Specifications....34
6.1.2 Related RTL Design Sherpa Documentation....34
6.1.3 RTL Implementation Files....34
6.1.4 Testbench and Verification Files....35
6.1.5 Tools and Frameworks....35
6.1.6 Design Patterns and Standards....36
6.1.7 External Resources....36
6.1.8 Version Control and Issue Tracking....36
6.1.9 Change History....36
6.1.10 Related Components....36
7 APB PIT 8254 - Block Hierarchy....38
7.1 Overview....38
7.2 Ports....38
7.2.1 Interface Summary....38

7.3 Functional Description....40
7.3.1 Dataflow Between Blocks....40
7.3.2 Figure 2.1: APB PIT 8254 Register Write and Read Paths....41
7.3.3 Block Responsibilities....41
7.3.4 Signal Flow Examples....42
7.4 Timing....44
7.4.1 Clock Domain Considerations....44
7.5 Design Notes....44
7.5.1 Reset Behavior Flow....44
8 APB PIT 8254 - Top-Level Interface....46
8.1 Overview....46
8.2 Parameters....46
8.3 Ports....47
8.3.1 Module Declaration....47
8.3.2 Signal Groups....48
8.4 Functional Description....49
8.4.1 Address Map....49
8.4.2 Error Response....50
8.5 Timing....50
8.5.1 Clock Domain Configuration....50
8.5.2 Reset Requirements....51
8.5.3 APB Protocol Timing....51
8.6 Usage Example....52
8.6.1 Single Clock Integration....52

8.6.2 Dual Clock Integration....53
9 APB PIT 8254 - Initialization and Programming Guide....54
9.1 Power-On Initialization Sequence....54
9.2 Complete Initialization Function....56
9.3 Usage Example....57
9.4 Runtime Configuration Changes....58
9.5 Common Initialization Errors....58
9.6 Debugging Initialization Issues....59
10 APB PIT 8254 - Common Use Cases....61
10.1 Use Case 1: Simple Timeout Timer....61
10.2 Use Case 2: RTOS System Tick....63
10.3 Use Case 3: Multiple Independent Timers....65
10.4 Use Case 4: Performance Profiling....67
10.5 Use Case 5: Watchdog Timer....69
10.6 Best Practices Summary....71
11 APB PIT 8254 - Register Map....73
11.1 Overview....73
11.2 PIT_CONFIG (0x000) - Global Configuration....73
11.3 PIT_CONTROL (0x004) - Control Word....74
11.4 PIT_STATUS (0x008) - Status Readback....75
11.5 COUNTERx_DATA (0x010, 0x014, 0x018) - Counter Values....76
11.6 Timing....77
11.7 Usage Example....78
12 8254 PIT (Programmable Interval Timer) - APB Implementation....80

12.1 Overview....80
12.2 Features....80
12.2.1 GATE (mode 0)....81
12.2.2 Counting-block priority....81
12.2.3 Counter latch....81
12.2.4 Byte-strobed loads....81
12.3 Architecture....82
12.3.1 Module Hierarchy....82
12.3.2 Parameters....82
12.4 Register Map....83
12.5 Counter Modes....83
12.6 Interrupt Outputs....84
12.7 Example Instantiation....84
12.8 Files....84
12.9 Development Status....85
12.10 Stated deviations from the 8254....85
12.11 Compliance....85

List of Figures

Figure 1.1: APB PIT 8254 Block Diagram....12
Figure 1.2: APB PIT 8254 High-Level Block Diagram....18
Figure 1.3: APB PIT 8254 Register Write and Read Paths....20
Figure 1.4: APB PIT 8254 Clock Configuration....22
Figure 1.5: APB PIT 8254 Clock Configuration....24
Figure 2.1: APB PIT 8254 Register Write and Read Paths....41

List of Tables

No tables in this document.

List of Waveforms

Waveform 1.1: Mode 0 Terminal Count.....14
Waveform 1.2: Mode 2 Rate Generator (reference only - not implemented).....14
Waveform 1.3: Mode 3 Square Wave Generator (reference only - not implemented).....15
Waveform 1.4: Gate Control.....15
Waveform 1.5: Readback Command (reference only - not implemented).....16

1 Pit 8254 Mas Index

Generated: 2026-10-13

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

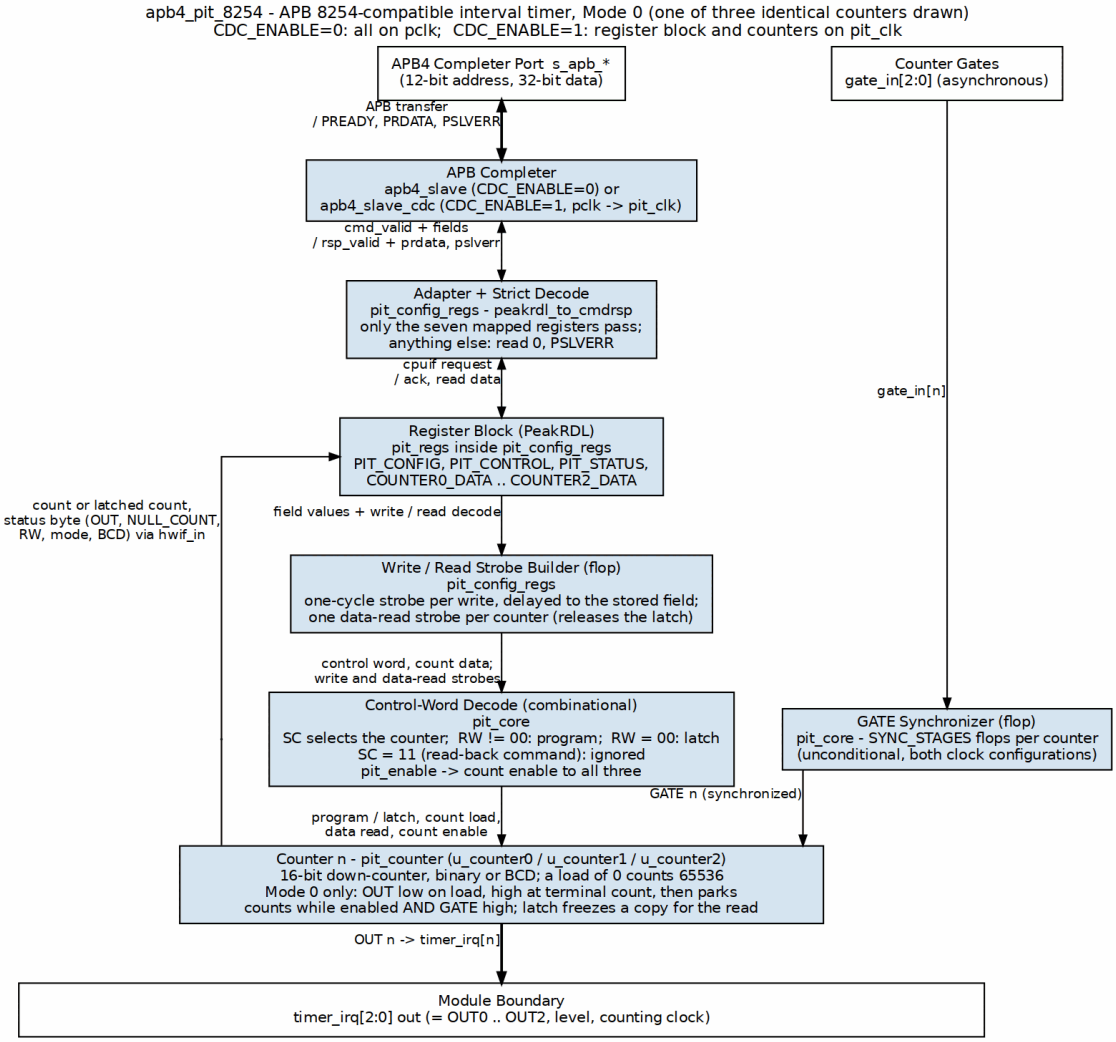
---

2 APB PIT 8254 - Overview

2.1 Overview

The APB Programmable Interval Timer (PIT 8254) is an 8254-compatible timer peripheral for precise interval timing and event generation in embedded systems. You get 3 independent 16-bit hardware counters running Mode 0 (Interrupt on Terminal Count), sitting behind an APB interface, with optional clock domain crossing if your timer ticks in its own clock domain.

2.1.1 Figure 1.1: APB PIT 8254 Block Diagram



APB PIT 8254 Block Diagram

2.2 Functional Description

2.2.1 Key Features

- **Three Independent Counters**: Three fully independent 16-bit down-counters
- **16-bit Count Values**: Each counter supports counts from 1 to 65,535 (1 to 9,999 in BCD), and a count of 0 means 65,536 (10,000 in BCD) – the 8254 convention, terminal count after a full wrap
- **Mode 0 Implementation**: Interrupt on terminal count (one-shot operation)
- **Binary Counting**: Standard binary countdown (BCD implemented but not yet tested)
- **GATE Control**: Individual GATE inputs; in Mode 0 a low GATE pauses the count and a high GATE resumes it from where it stopped, through a SYNC_STAGES synchronizer
- **OUT Signals**: Individual OUT outputs indicating terminal count reached
- **APB Interface**: Standard AMBA APB4 compliant register interface
- **Clock Domain Crossing**: Optional CDC support for independent APB and timer clocks
- **PeakRDL Integration**: Register map generated from SystemRDL specification
- **Counter Latch**: A control word with RW=00 freezes the selected count for an atomic read, the 8254 way
- **Status Readback**: Per-counter status including mode, RW mode, NULL_COUNT, and OUT state
- **Control Word Programming**: 8254-compatible control word format

2.2.2 Applications

Real-Time Operating Systems: - Periodic tick generation for RTOS schedulers - Timeout implementation - Task deadline enforcement - System time tracking

Performance Profiling: - Code execution timing - Event interval measurement - Timeout detection - Profiling counters

Multi-Rate Timing: - Multiple simultaneous timing domains - Independent periodic tasks - Asynchronous event generation - Programmable delay generation

Legacy System Compatibility: - PC/AT timer emulation - Retro system peripherals - Sound generation base timer - Speaker control timing

2.2.3 Design Philosophy

8254 Compatibility: The PIT follows the 8254 specification for control word format, counter behavior, and status readback. It is not a cycle-exact clone – it holds functional compatibility for Mode 0 operation, and where it deviates, this document says so out loud.

Modern Integration: The original 8254 hangs off separate port I/O addresses. This implementation uses a unified APB register interface instead, which is what you want for modern SoC integration.

Reliability: A 30-test suite (gate, func and full levels, both CDC configurations) validates the core functionality, including a dedicated regression for every issue #52 finding. The design includes proper clock enable gating and readback paths.

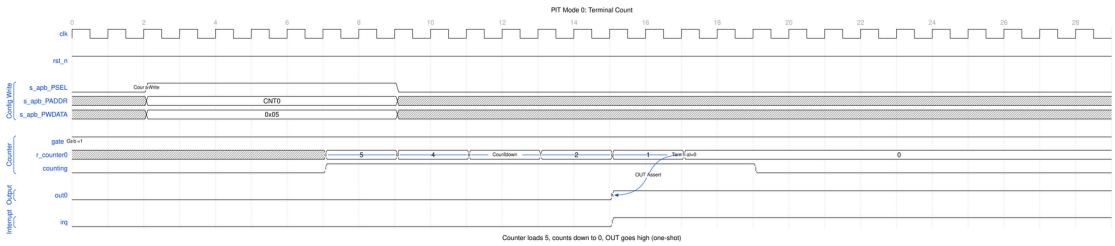
Standards Compliance: - **APB Protocol**: Full AMBA APB4 specification compliance - **PeakRDL**: Industry-standard SystemRDL for register generation - **Reset Convention**: Consistent active-low asynchronous reset (`presetn`)

Reusability: Clean module hierarchy and well-defined interfaces make integration straightforward. Optional CDC support gives you flexible clock domain configuration without design changes.

2.3 Waveforms

2.3.1 Waveform 1.1: Mode 0 Terminal Count

In Mode 0, the counter counts down from the loaded value and asserts OUT when reaching zero.

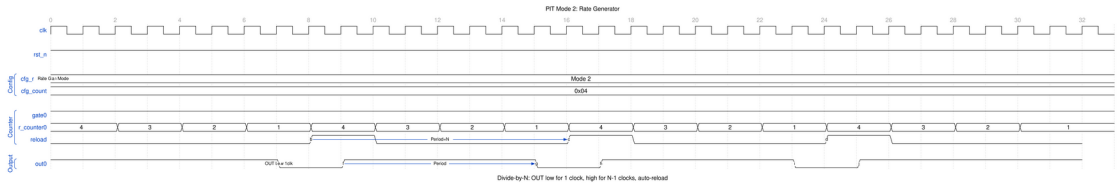


PIT Mode 0 Terminal Count

The counter loads with the programmed value and decrements on each clock. When terminal count (0) is reached, OUT goes high and remains high until a new count is loaded.

2.3.2 Waveform 1.2: Mode 2 Rate Generator (reference only - not implemented)

Mode 2 on a real 8254 produces a divide-by-N clock output. This waveform is 8254 reference behavior: the RTL implements Mode 0 only, and a control word selecting Mode 2 still yields Mode 0 counting (see Known Limitations).

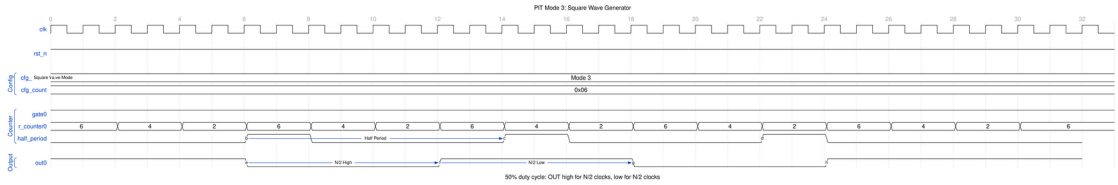


PIT Mode 2 Rate Generator

OUT is normally high, going low for one clock when the counter reaches 1. The counter auto-reloads, creating a periodic pulse train.

2.3.3 Waveform 1.3: Mode 3 Square Wave Generator (reference only - not implemented)

Mode 3 on a real 8254 produces a 50% duty cycle square wave. As with Mode 2, this is reference behavior only - the RTL runs Mode 0 regardless of the programmed mode.

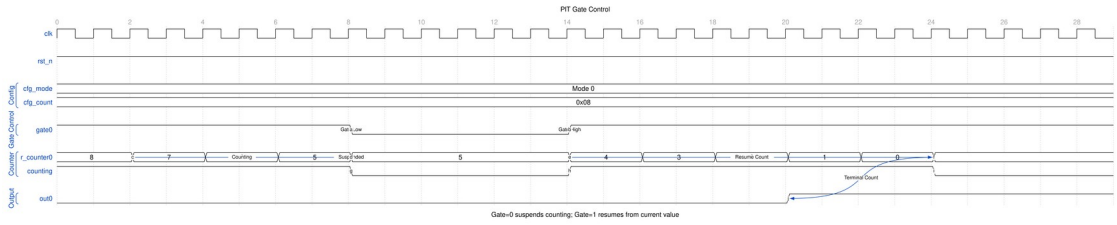


PIT Mode 3 Square Wave

OUT toggles every N/2 clocks, producing a symmetric square wave output.

2.3.4 Waveform 1.4: Gate Control

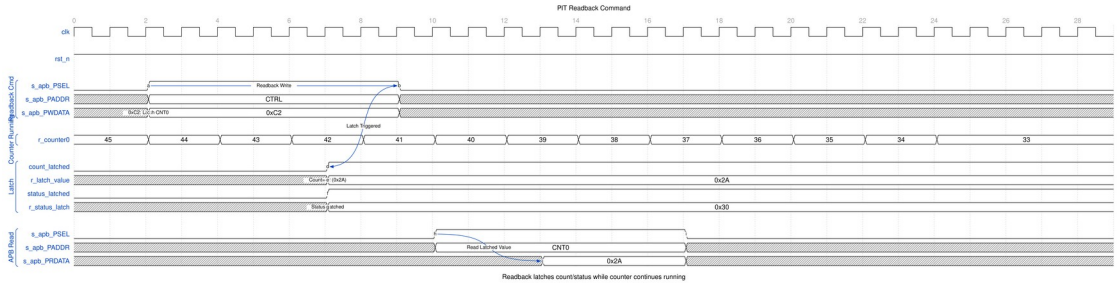
In Mode 0 GATE is an enable, not a trigger: while it is low the counter holds its value, and when it goes high again counting resumes from that value with no reload. A load lands whatever GATE is doing – only the decrement is gated. GATE reaches the counter through a SYNC_STAGES-flop synchronizer (default 2), so a transition takes effect two counting clocks after the pin moves. This is the 8254 behaviour, and since the issue #52 fixes (2026-09-09) it is what the RTL does.



PIT Gate Control

2.3.5 Waveform 1.5: Readback Command (reference only - not implemented)

On a real 8254, the readback command (SC=11) latches counter value and status while the counter continues running. In the delivered RTL, SC=11 is a NO-OP: status is always live in PIT_STATUS, and the count is latched through the ordinary counter-latch command instead (a control word with RW=00, released by the next COUNTERx_DATA read or by reprogramming the counter – see ch05). The waveform shows the reference behavior for the read-back command itself.



PIT Readback

2.4 Design Notes

2.4.1 Comparison with 8254

The APB PIT 8254 is architecturally compatible with the 8254 but has key differences:

Feature	8254	APB PIT 8254
Interface	Port I/O (8-bit)	AMBA APB4 (32-bit)
Counter Count	3	3 (fixed)
Counter Size	16-bit	16-bit
Modes	0-5	Mode 0 only (currently)
BCD Counting	Supported	Implemented, not tested
Read/Write	Byte-by-byte on D7-D0	16-bit word, or one byte on its own lane: RW=01 in [7:0], RW=10 in [15:8] (write and read)
Latch Command	Supported	Supported: control word with RW=00 latches; the next data read returns the value and releases, and a reprogram or reload releases it too (see

Feature	8254	APB PIT 8254
		ch05)
Read-Back Command	Supported	No-op (SC=11 ignored; status bytes are always live)
Clock Source	External CLK pins	Configurable (`pit_clk`)
Integration	Standalone chip	SoC peripheral block

2.4.2 Design Scope

Currently Implemented: - Mode 0 (Interrupt on Terminal Count) - Binary counting - Control word programming - Counter data writes - Status readback - Counter latch command - GATE pause/resume with input synchronizer - OUT signal generation - Optional clock domain crossing

Not Yet Implemented: - Modes 1-5 (Retriggerable One-Shot, Rate Generator, Square Wave, etc.) - Full read-back command support - BCD counting verification

Implementation Quality: - **Validated** for Mode 0 operation (30 tests, both CDC configs; one test-tolerance gap, see the index) - **Issue #52 closed** (fixed 2026-09-09) with a regression test per finding - **Well-Documented** RTL and verification - **FPGA Verified** on Verilator simulation

---

Version: 1.1 **Last Updated:** 2026-09-09 **Status:** RTL Functional (Mode 0; issue #52 fixed 2026-09-09; see the index)

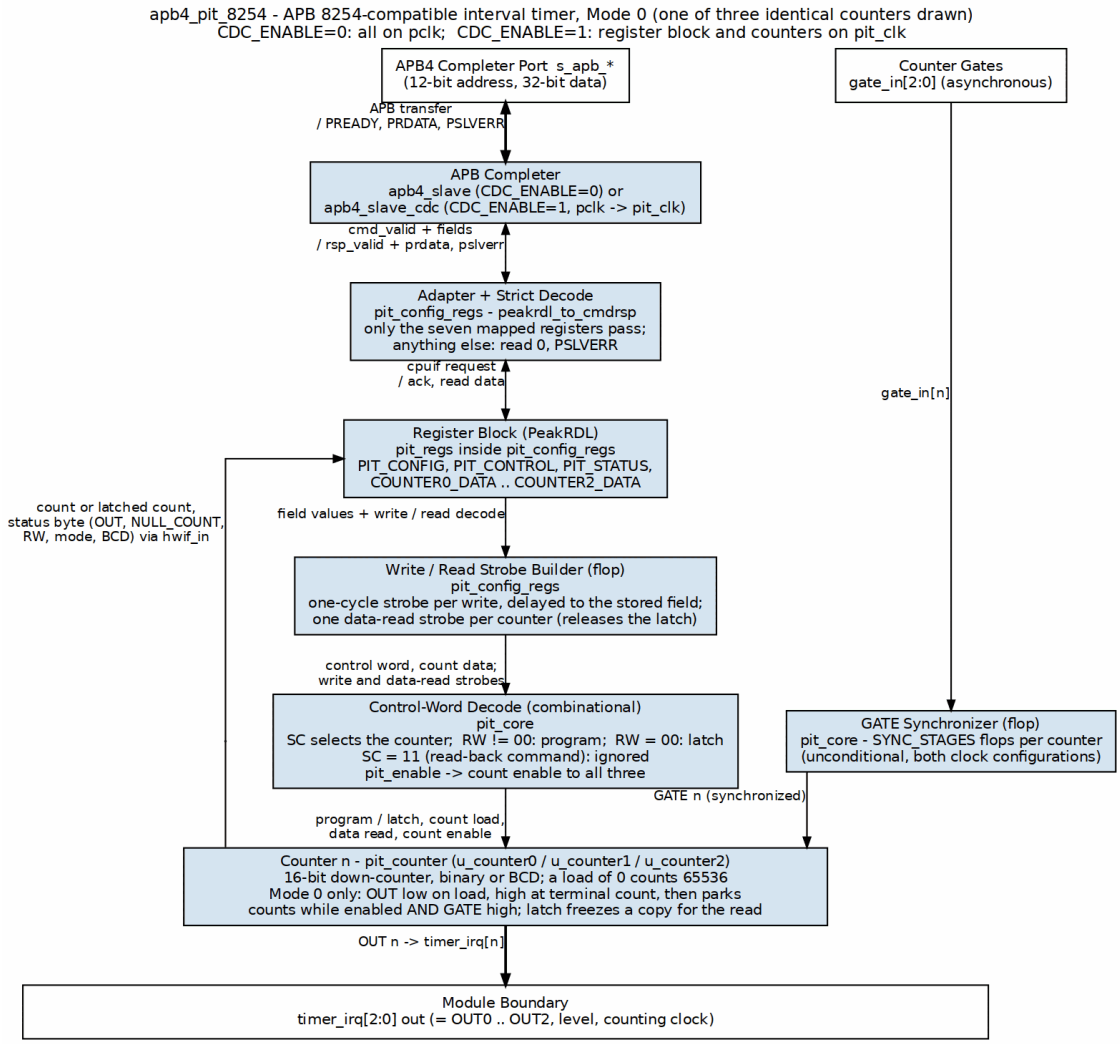
RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

3 APB PIT 8254 - Architecture

3.1 Overview

3.1.1 Figure 1.2: APB PIT 8254 High-Level Block Diagram



APB PIT 8254 High-Level Block Diagram

3.1.2 Module Hierarchy

```
apb4_pit_8254
├── apb4_slave (CDC_ENABLE=0) or apb4_slave_cdc (CDC_ENABLE=1)
│   └── Converts APB protocol to cmd/rsp interface
├── pit_config_regs
│   ├── peakrdl_to_cmdrsp (protocol adapter)
│   └── pit_regs (PeakRDL generated)
```

```
│       └── Register file with hwif interface
└── pit_core
    ├── pit_counter (Counter 0)
    ├── pit_counter (Counter 1)
    └── pit_counter (Counter 2)
```

3.1.3 Three-Layer Architecture

Following the HPET design pattern, the PIT uses a clean three-layer architecture:

Layer 1: APB Interface (apb4_pit_8254) - Protocol conversion (APB → cmd/rsp) - Optional clock domain crossing - Top-level integration - Parameter configuration

Layer 2: Configuration Registers (pit_config_regs) - Register file integration - Strict address decode: only the seven mapped registers are visible, everything else in the 4 KB window is dropped with PSLVERR - One-cycle command strobes, aligned to the stored field value so a load takes the byte-strobe-merged register contents - Counter readback connection and data-read strobes for the latch - Status feedback aggregation

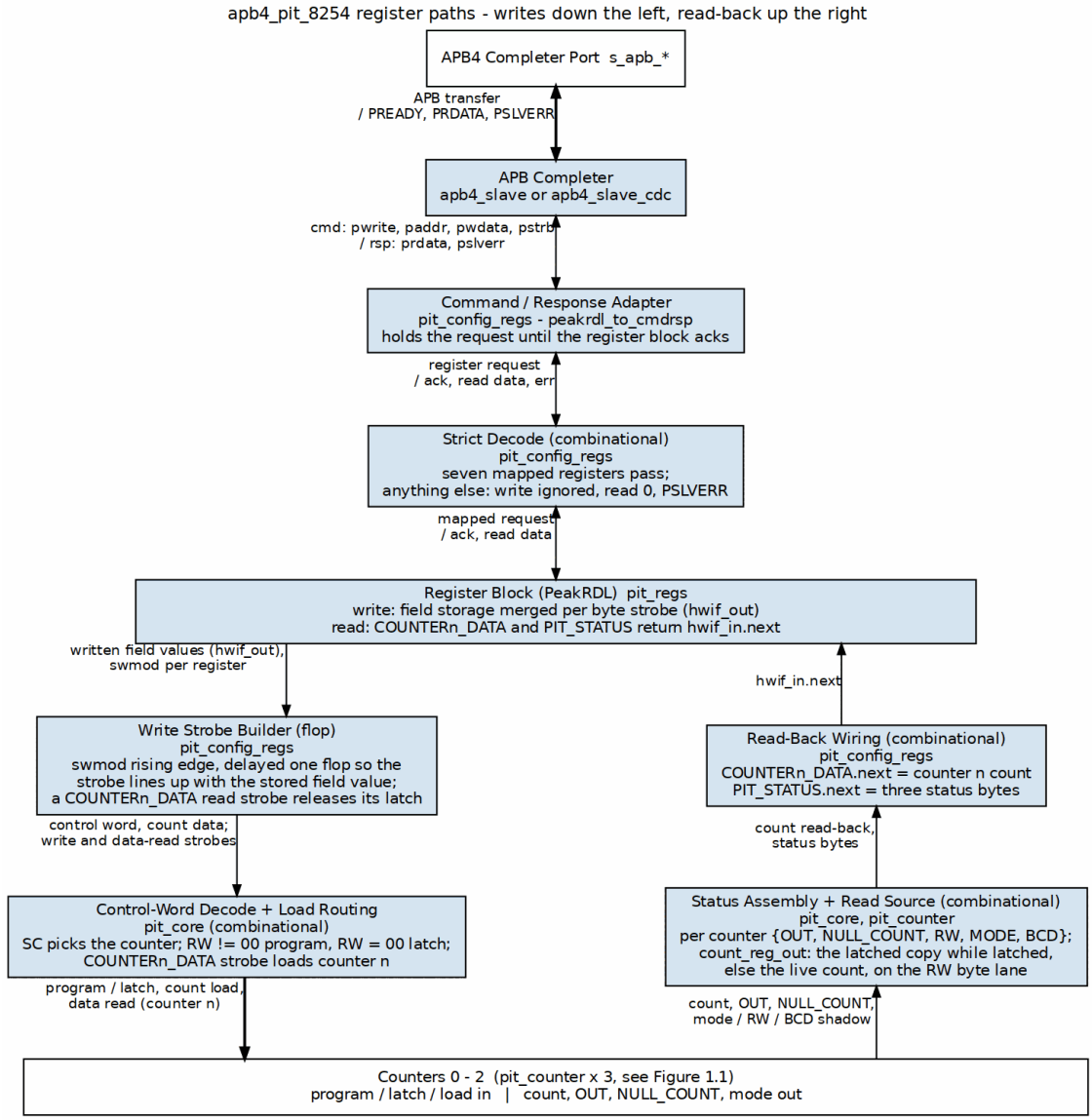
Layer 3: Core Logic (pit_core + pit_counter) - Control-word split: RW != 00 programs the selected counter, RW = 00 latches it - Counter control and data routing - Three independent pit_counter instances - Mode 0 counting logic - GATE synchronizer (SYNC_STAGES flops) and OUT signal management

3.2 Functional Description

3.2.1 Data Flow

A register write travels down the left of Figure 1.3: the PeakRDL field stores the byte-strobe-merged value, the strobe builder fires one aligned strobe, and pit_core routes it to the selected counter. A read climbs the right: the counter's count (or its latched copy) and status byte are wired into `hwif_in`, and the register block returns them on the APB response.

3.2.2 Figure 1.3: APB PIT 8254 Register Write and Read Paths



APB PIT 8254 Register Write and Read Paths

3.2.3 Counter Control State

There is no explicit state machine in `pit_counter`. Counter control is two flags plus the count itself:

- **`r_null_count`** - set at reset and by every control-word program of this counter, cleared by a count load. It reads back as the NULL_COUNT status bit.

- **`r_counting`** - set by a count load, cleared when the decremented count reaches zero (terminal count, OUT goes high). Nothing else sets it: there is no re-arm path, so after terminal count the counter sits in one steady state (`r_counting=0`, count 0, OUT high) until the next load.

A cycle actually counts when `r_counting`, the PIT enable and the synchronized GATE are all high – that single condition is the whole GATE story in Mode 0.

Behavior over a Mode 0 cycle:

1. Reset: `r_null_count=1`, `r_counting=0`, `OUT=0`.
2. Count load: count captured in one cycle (no intermediate value), `r_null_count` cleared, OUT driven low, `r_counting` set. The load is not gated by GATE or by the PIT enable – only the decrement is.
3. Counting: decrement on each clock while the PIT is enabled and GATE is high. GATE low pauses the count where it stands; GATE high resumes it from that value with no reload.
4. Terminal count: detected on the decremented value, so a load of N counts N clocks and a load of 0 counts 65536 (10000 in BCD). OUT goes high and `r_counting` clears; the count parks at 0 and OUT stays high until the next load drives it low again.

3.2.4 Control Flow

Counter Programming Sequence: 1. Write `PIT_CONTROL` with control word (counter select, mode, RW mode) 2. Control word decoded and routed to selected counter (RW != 00 programs it; RW = 00 is the latch command and leaves the programming alone) 3. Write `COUNTERx_DATA` with the count on the lane the RW mode selects 4. Counter loads the value and counts whenever GATE is high and the PIT is enabled 5. Counter decrements on each clock cycle 6. When count reaches 0, OUT goes high

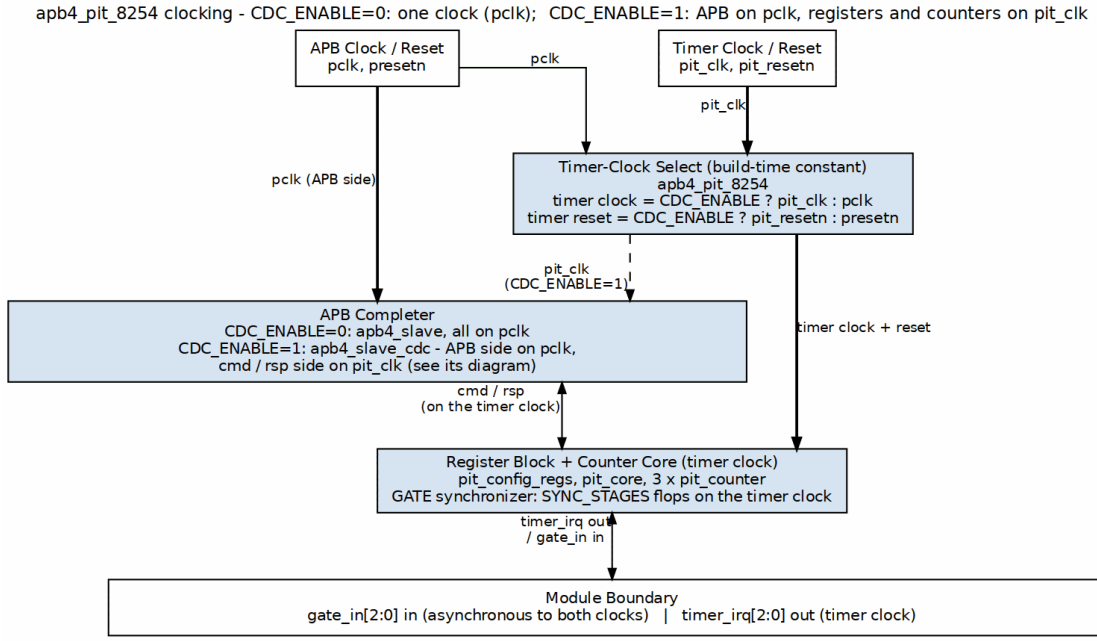
Status Readback: 1. Read `PIT_STATUS` register 2. Returns 3 bytes (one per counter) with packed status fields 3. Status includes: OUT state, NULL_COUNT, RW mode, counter mode, BCD flag

3.3 Timing

3.3.1 Clock Domains

With `CDC_ENABLE=0` everything runs on `pclk`. With `CDC_ENABLE=1` the APB side stays on `pclk`, `apb4_slave_cdc` carries the command/response stream across, and the register block and the counters run on `pit_clk`.

3.3.2 Figure 1.4: APB PIT 8254 Clock Configuration



APB PIT 8254 Clock Configuration

`gate_in` is a pin and is asynchronous to the counting clock in both modes, so it always passes a SYNC_STAGES-flop synchronizer (default 2) inside `pit_core` before it reaches the counters: a GATE transition takes effect SYNC_STAGES counting clocks after the pin moves, in either configuration.

3.4 Design Notes

3.4.1 Reset Behavior

Power-On Reset: - All counters: NULL_COUNT=1, counting=0, OUT=0 - PIT disabled (PIT_CONFIG=0) - All count values cleared

Soft Reset (PIT disable): - Counting pauses (the clock enable is gated; the internal counting flag is NOT cleared, so counting resumes where it left off on re-enable) - Count values preserved - OUT signals remain in current state - NULL_COUNT flags unchanged

---

Version: 1.1 **Last Updated:** 2026-09-09

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

4 APB PIT 8254 - Clocks and Reset

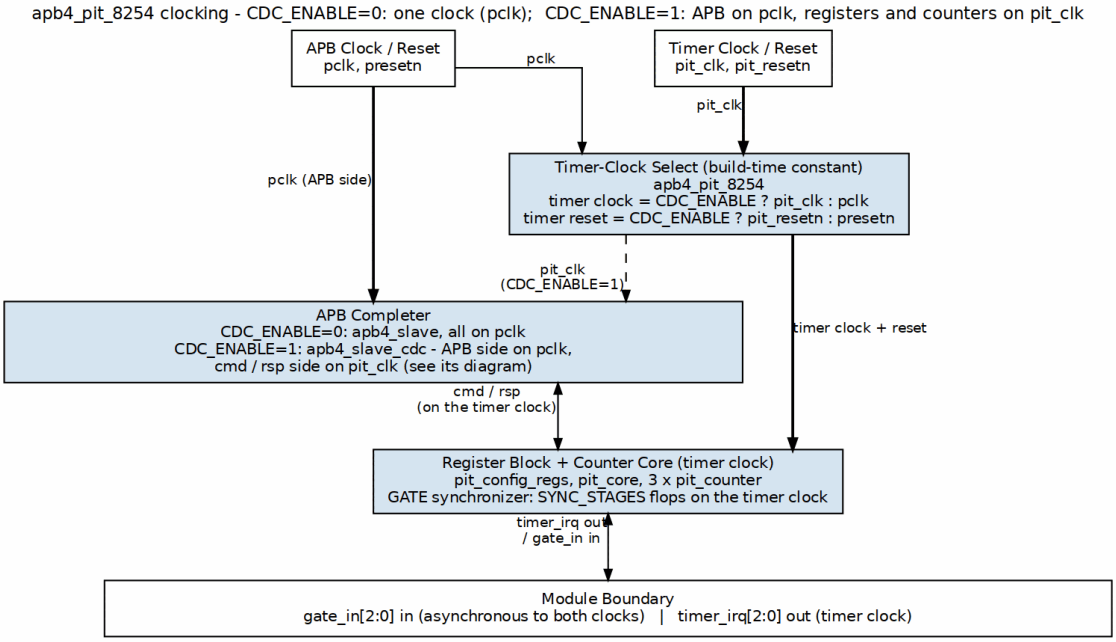
4.1 Overview

The APB PIT 8254 supports two clock domain configurations controlled by the `CDC_ENABLE` parameter. Pick the single-clock variant when your timer can run off the APB clock; pick the dual-clock variant when the timer needs its own timebase.

Single Clock Configuration (CDC_ENABLE=0): `pclk` clocks the APB completer, the register file and the counters.

Dual Clock Configuration (CDC_ENABLE=1): `pclk` clocks the APB side of `apb4_slave_cdc`; `pit_clk` clocks its command/response side, the register file and the counters.

4.1.1 Figure 1.5: APB PIT 8254 Clock Configuration



APB PIT 8254 Clock Configuration

4.2 Ports

4.2.1 Clock Signals

pclk (APB Clock): - **Purpose:** APB bus interface timing - **Frequency:** Determined by system APB bus (typically 50-100 MHz) - **Domain:** Always present, required for register access - **Usage:** Clocks APB slave, optional CDC logic

pit_clk (PIT Timer Clock): - **Purpose:** Counter decrement timing - **Frequency:** Configurable, determines timer resolution - **Domain:** Only used when `CDC_ENABLE=1` - **Usage:** Clocks counter logic when separate from APB domain - **Note:** When `CDC_ENABLE=0`, counters use `pclk` directly

4.3 Functional Description

4.3.1 Clock Enable Signal

i_clk_en (Clock Enable): - **Purpose:** Global enable/disable for counter operation - **Source:** `PIT_CONFIG.PIT_ENABLE` register bit - **Effect:** Gates counter decrement logic without stopping the clock - **Behavior:** - `i_clk_en=0`: Counters hold current value - `i_clk_en=1`: Counters decrement normally (if GATE high)

Implementation:

```
// Counter decrement with clock enable check
if (r_counting && i_clk_en) begin
    if (r_count == 16'd0) begin
        r_out <= 1'b1;        // Terminal count reached
        r_counting <= 1'b0;
    end else begin
        r_count <= r_count - 16'd1;
    end
end
```

Benefits: - Software-controlled start/stop without glitches - Preserves counter values during disable - No clock domain crossing issues - Synchronous control

4.3.2 Reset Signal

presetn (Active-Low Asynchronous Reset): - **Type:** Asynchronous assertion, synchronous deassertion (hand-written logic). The generated register file (pit_regs.sv) is the exception: it resets SYNCHRONOUSLY (plain always_ff with rst = ~rst_n), so its fields do not reset until a clock edge arrives - **Polarity:** Active-low (standard APB convention) - **Domain:** Applied to all clock domains - **Purpose:** Initialize all state to known values

That synchronous-reset exception in the generated file is the kind of detail that only bites you once – after that you check for it in every PeakRDL block you integrate.

Power-On Reset:

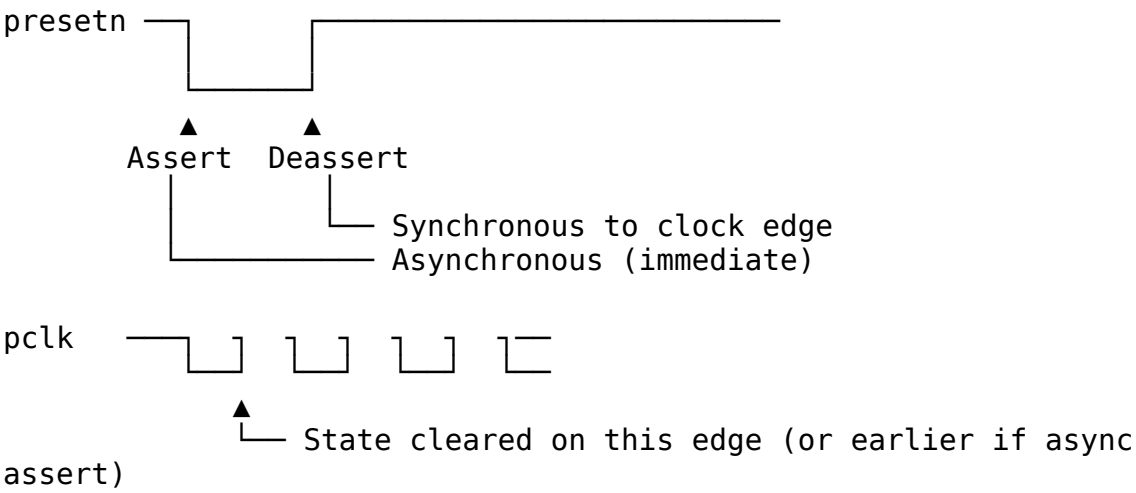
```
// All counters reset to safe state
r_count         <= 16'd0;       // Counter value cleared
r_null_count    <= 1'b1;        // No count loaded flag set
r_counting      <= 1'b0;        // Not counting
r_out           <= 1'b0;        // OUT signal low
r_reload_pending<= 1'b0;        // No reload pending
r_mode          <= 3'b000;      // Mode 0
```

```
r_rw_mode       <= 2'b00;       // No access mode set
r_bcd           <= 1'b0;        // Binary mode
```

Register Reset Values:

```
PIT_CONFIG    = 0x00000000  (PIT disabled)
PIT_STATUS    = 0x00404040  (all counters NULL_COUNT=1, OUT=0)
COUNTER0_DATA = 0x00000000  (no count loaded)
COUNTER1_DATA = 0x00000000  (no count loaded)
COUNTER2_DATA = 0x00000000  (no count loaded)
```

Reset Timing:



Reset Sequence:

1. **Assertion (asynchronous):**
   - Immediately forces all state to reset values
   - No clock edge required
   - Occurs as soon as `presetn=0`
2. **Hold Period:**
   - Must hold `presetn=0` for minimum 10 clock cycles of the slowest clock (matches the top-level interface chapter)
   - Ensures all registers properly reset
   - Applies to slowest clock domain (if CDC enabled)
3. **Deassertion (synchronous):**
   - Released synchronously with clock edge
   - State begins normal operation
   - Counters remain idle until configured

Recommended Reset Sequence:

```
// 1. Assert reset
set_reset(0);
wait_us(10);  // Hold for sufficient time

// 2. Deassert reset
set_reset(1);
wait_us(10);  // Allow settling

// 3. Verify reset state
assert(read_register(PIT_CONFIG) == 0x00000000);
assert(read_register(PIT_STATUS) == 0x00404040);

// 4. Configure PIT
write_register(PIT_CONTROL, 0x30);  // Counter 0, Mode 0
write_register(COUNTER0_DATA, 1000);
write_register(PIT_CONFIG, 0x01);   // Enable PIT
```

4.4 Timing

4.4.1 Clock Domain Crossing (CDC)

When CDC_ENABLE=1:

The design includes clock domain crossing infrastructure to safely transfer data between the APB and timer clock domains:

CDC Components: - `apb4_slave_cdc` module provides safe crossing from `pclk` to `pit_clk` - Command/response interface synchronized using gray-code FIFOs - Handshaking ensures no data loss across domains - Status signals synchronized back to `pclk` domain

CDC Timing:

```
APB Write → pclk domain → CDC FIFO → pit_clk domain → Register Update
                (1-2 cycles)        (2-3 cycles)      (1 cycle)
Total latency: 4-6 pit_clk cycles
```

CDC Verification: - All 6/6 tests pass with `CDC_ENABLE=1` configuration - No metastability issues observed in simulation - Proper handshaking verified with stress tests

When CDC_ENABLE=0:

The design uses a single clock domain with `apb4_slave` module (no CDC):

Direct Connection: - APB slave converts APB protocol to cmd/rsp interface - No domain crossing required - Lower latency (2-3 cycle register access)

Single Clock Timing:

```
APB Write → pclk domain → Register Update
                (1-2 cycles)
Total latency: 2-3 pclk cycles
```

4.4.2 Multi-Clock Timing Constraints

For CDC_ENABLE=1 configurations, apply these timing constraints:

Clock Definitions:

```
create_clock -period 10.0 [get_ports pclk]
create_clock -period 20.0 [get_ports pit_clk]
```

Asynchronous Clock Groups:

```
set_clock_groups -asynchronous \
    -group [get_clocks pclk] \
    -group [get_clocks pit_clk]
```

CDC Path Constraints:

```
# CDC paths handled by apb4_slave_cdc module
# Verify no timing paths between domains except through CDC
set_false_path -from [get_clocks pclk] -to [get_clocks pit_clk]
set_false_path -from [get_clocks pit_clk] -to [get_clocks pclk]
```

Reset Synchronization:

```
# Reset must be synchronized to each clock domain
set_false_path -from [get_ports presetn] -to [all_registers]
```

4.5 Design Notes

4.5.1 Clock Gating

Dynamic Clock Gating:

The PIT does NOT implement dynamic clock gating at the module level. If you want clock gating, implement it at the integration level:

Integration-Level Gating Example:

```
// External clock gate (system integrator's responsibility)
logic gated_pclk;
assign gated_pclk = pclk & pit_clock_enable;

apb4_pit_8254 #(
    .CDC_ENABLE(0)
) u_pit (
    .pclk       (gated_pclk),  // Gated clock
```

```
    // ...
);
```

Static Clock Enable:

When `PIT_ENABLE=0`, counters use clock enable gating internally: - Clock still toggles (no clock tree gating) - Counter logic uses `if (i_clk_en)` conditions - Reduces dynamic power by preventing state changes - No glitches or timing issues

---

Version: 1.0 **Last Updated:** 2025-11-08

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

5 APB PIT 8254 - Acronyms and Terminology

5.1 References

5.1.1 Acronyms

Acronym	Definition
AMBA	Advanced Microcontroller Bus Architecture
APB	Advanced Peripheral Bus
APB4	Advanced Peripheral Bus Protocol Version 4
ASIC	Application-Specific Integrated Circuit
BCD	Binary-Coded Decimal
BFM	Bus Functional Model
CDC	Clock Domain Crossing
CLK	Clock
CMD	Command
CPU	Central Processing Unit
CW	Control Word
DUT	Design Under Test
FIFO	First-In First-Out
FPGA	Field-Programmable Gate Array
FSM	Finite State Machine
HWIF	Hardware Interface
I/O	Input/Output
IRQ	Interrupt Request
LSB	Least Significant Byte
MSB	Most Significant Byte
OUT	Output signal (terminal count indicator)
PC/AT	Personal Computer/Advanced Technology

Acronym	Definition
PIT	Programmable Interval Timer
RDL	Register Description Language
RSP	Response
RTOS	Real-Time Operating System
RTL	Register Transfer Level
RW	Read/Write
SoC	System-on-Chip
SystemRDL	System Register Description Language
TB	Testbench
W1C	Write-1-to-Clear
WO	Write-Only

5.1.2 Terminology

8254 Compatibility: The 8254 Programmable Interval Timer is the original reference specification. APB PIT 8254 maintains functional compatibility for Mode 0 operation while adapting the interface from port I/O to APB protocol.

Active-Low Reset: A reset signal that performs reset when driven to logic 0. The APB PIT uses `presetn` (active-low) following standard APB convention.

APB Protocol: AMBA APB4 is a simple synchronous protocol for low-bandwidth peripheral access. APB transactions consist of: - **Setup phase**: `psel=1`, `penable=0` - **Access phase**: `psel=1`, `penable=1` - **Response**: `pready=1` when slave completes transaction

BCD Counting: Binary-Coded Decimal mode where each 4-bit nibble represents 0-9. For a 16-bit counter, BCD mode counts 1 to 9999 (4 decimal digits) and a load of 0 counts 10000, the BCD counterpart of the binary 0-means-65536 rule (the counter borrows from 0x0000 to 0x9999 and reaches terminal count on the decremented value).

Binary Counting: Standard binary mode where the full 16-bit range is used: a load of N counts N, and a load of 0 counts 65,536.

Clock Domain Crossing (CDC): Transfer of signals between two asynchronous clock domains. When `CDC_ENABLE=1`, the APB PIT includes synchronization logic to safely cross between `pclk` (APB clock) and `pit_clk` (timer clock).

Clock Enable: A signal (`i_clk_en`) that gates counter operation without stopping the clock. When `i_clk_en=0`, counters hold their current value.

Control Word: An 8-bit value written to `PIT_CONTROL` register to configure counter operation. Format follows 8254 specification:

```
[7:6] SC   - Counter Select (00=Counter 0, 01=Counter 1, 10=Counter 2)
[5:4] RW   - Read/Write mode (00=counter latch command, 01=LSB only,
10=MSB only, 11=LSB then MSB)
[3:1] MODE - Counter mode (000=Mode 0, 001-101=Modes 1-5)
[0]   BCD  - 0=Binary, 1=BCD
```

Counter: A 16-bit down-counter that decrements on each clock cycle when enabled. The PIT contains three independent counters (Counter 0, Counter 1, Counter 2).

GATE Input: Per-counter count enable, as on the 8254 in Mode 0: while `gate_in[N]` is low the counter holds its value, and when it goes high again counting resumes from that value with no reload. A load is not gated. The pin passes a SYNC_STAGES-flop synchronizer (default 2) on the counting clock, so a transition takes effect two clocks after it happens.

Interrupt on Terminal Count (Mode 0): Counter operation mode where OUT signal goes high when count reaches zero, typically used to generate interrupts.

LSB/MSB Access: 8254 compatibility feature for byte-by-byte access. Each byte lives on its own lane of COUNTERx_DATA for both write and read: - **LSB only (RW=01)**: the low byte, in bits [7:0]; a load leaves the high byte 0 - **MSB only (RW=10)**: the high byte, in bits [15:8]; a load leaves the low byte 0, and a read returns 0 in [7:0] - **LSB then MSB (RW=11)**: Full 16-bit access (recommended for APB)

NULL_COUNT Flag: Status bit indicating no count value has been loaded into a counter. Set to 1 on reset, cleared to 0 when count value written.

OUT Signal: Per-counter output signal indicating terminal count reached. In Mode 0: - `OUT=0` during counting - `OUT=1` when count reaches zero

PeakRDL: An open-source toolchain for generating register RTL from SystemRDL descriptions. Used to generate the PIT register file from `pit_regs.rdl`.

Reload Value: The count value written to `COUNTERx_DATA` register. Counter loads this value and begins counting down.

Terminal Count: The state when a counter reaches zero. In Mode 0, this sets the OUT signal high and stops counting.

Timer Interrupt: In this implementation, `timer_irq[N]` outputs are driven by the corresponding OUT signals, providing interrupt capability for system integration.

5.1.3 Register Field Access Types

RO (Read-Only): Software can read this field, but writes have no effect. Hardware controls the value.

WO (Write-Only): Software can write this field, but reads return undefined or zero values. Used for command registers.

RW (Read-Write): Software can read and write this field. Hardware may also update the value (e.g., counter readback).

W1C (Write-1-to-Clear): Software writes 1 to clear the bit, writes 0 have no effect. Used for interrupt/status flags. (Note: Not used in PIT, but common in other peripherals)

5.1.4 SystemRDL Concepts

hwif (Hardware Interface): The interface between PeakRDL-generated register file and custom RTL logic. Provides: - `hwif_out`: Register values from SW writes (to hardware) - `hwif_in`: Hardware values to read back (from hardware)

regfile: SystemRDL construct representing a collection of related registers.

field: Smallest addressable unit within a register, representing specific bits with defined access semantics.

5.1.5 Design Architecture Terms

Three-Layer Architecture: The PIT uses a clean separation: 1. **APB Interface Layer** - Protocol conversion and optional CDC 2. **Configuration Layer** - Register file and edge detection 3. **Core Logic Layer** - Counter implementation

Edge Detection: Converting a level signal (register field) to a pulse (write strobe). Critical for triggering actions on register writes without level-sensitive issues.

cmd/rsp Interface: Internal protocol used between APB slave and register adapter: - **cmd**: Command signals (address, write data, write enable) - **rsp**: Response signals (read data, valid)

Version: 1.0 **Last Updated:** 2025-11-08

---

6 APB PIT 8254 - References

6.1 References

6.1.1 Primary Specifications

8254 Programmable Interval Timer - **Document**: 8254 Datasheet - **Relevance**: Original reference design for PIT functionality and control word format - **Note**: APB PIT 8254 implements Mode 0 with APB interface adaptation

AMBA APB Protocol Specification - **Document**: ARM IHI 0024C - AMBA APB Protocol Specification v2.0 - **Publisher**: ARM Limited - **Relevance**: Defines APB4 protocol used for register access - **URL**: https://developer.arm.com/documentation/ihi0024/latest/

SystemRDL Specification - **Document**: SystemRDL 2.0 Language Reference Manual - **Publisher**: Accellera Systems Initiative - **Relevance**: Register description language used for PIT register definition - **URL**: https://www.accellera.org/downloads/standards/systemrdl

6.1.2 Related RTL Design Sherpa Documentation

APB HPET Specification - **Location**: `projects/components/legacy-ip/retro_legacy_blocks/docs/hpet_mas/` - **Relevance**: Similar APB timer peripheral, shares architectural patterns and testbench infrastructure - **See**: HPET specification for examples of APB timer implementation

Retro Legacy Blocks PRD - **Location**: `projects/components/legacy-ip/retro_legacy_blocks/PRD.md` - **Relevance**: Master requirements for all RLB peripherals - **See**: Section on address map, integration strategy, and block priorities

Retro Legacy Blocks README - **Location**: `projects/components/legacy-ip/retro_legacy_blocks/README.md` - **Relevance**: Component overview, status table, and integration examples

Repository Root CLAUDE.md - **Location**: `/CLAUDE.md` - **Relevance**: Repository-wide design standards, coding conventions, tool usage - **See**: Sections on reset handling, FPGA synthesis, testbench architecture

Global Requirements Document - **Location**: `/GLOBAL_REQUIREMENTS.md` - **Relevance**: Mandatory requirements for all RTL and testbench code - **See**: Reset macro standards, FPGA attributes, array syntax, SRAM standards

6.1.3 RTL Implementation Files

Top-Level Module - **File**: `rtl/pit_8254/apb4_pit_8254.sv` - **Description**: APB interface wrapper with optional clock domain crossing

Core Logic - **File**: `rtl/pit_8254/pit_core.sv` - **Description**: Counter control and routing for three independent counters

Counter Module - **File**: `rtl/pit_8254/pit_counter.sv` - **Description**: Single counter implementation with Mode 0 support

Configuration Registers - **File**: `rtl/pit_8254/pit_config_regs.sv` - **Description**: Wrapper connecting PeakRDL registers to core logic

PeakRDL Generated Files - **Files**: - `rtl/pit_8254/pit_regs.sv` - Register file RTL - `rtl/pit_8254/pit_regs_pkg.sv` - Package definitions - **Source**: `rdl/pit_8254/pit_regs.rdl` - **Generated by**: PeakRDL regblock tool

RTL README - **File**: `rtl/pit_8254/README.md` - **Description**: Status, stated deviations, test results and the known test-vs-design gap

6.1.4 Testbench and Verification Files

Testbench Class - **File**: `dv/tbclasses/pit_8254/pit_tb.py` - **Description**: Main testbench providing APB BFM and helper methods

Basic Test Suite - **File**: `dv/tbclasses/pit_8254/pit_tests_basic.py` - **Description**: Six basic tests covering register access, enable/disable, Mode 0, status

Test Runner - **File**: `dv/tests/test_apb4_pit_8254.py` - **Description**: Pytest test runner invoking CocoTB tests with parameterization

Configuration - **File**: `dv/tests/conftest.py` - **Description**: Pytest configuration, logging setup, markers

6.1.5 Tools and Frameworks

PeakRDL - **Tool**: peakrdl-regblock - **Purpose**: Generate RTL register files from SystemRDL specifications - **Documentation**: https://github.com/SystemRDL/PeakRDL-regblock - **Usage**: `peakrdl regblock pit_regs.rdl --cpuif apb4 -o ../`

CocoTB - **Framework**: CocoTB Python testbench framework - **Purpose**: HDL verification using Python - **Documentation**: https://docs.cocotb.org/ - **Usage**: Provides `@cocotb.test()` decorator and simulation infrastructure

Verilator - **Tool**: Verilator HDL simulator - **Purpose**: Fast cycle-accurate RTL simulation - **Documentation**: https://verilator.org/ - **Usage**: Underlying simulator for CocoTB tests

pytest - **Framework**: Python testing framework - **Purpose**: Test discovery, parameterization, and reporting - **Documentation**: https://pytest.org/ - **Usage**: `pytest dv/tests/test_apb4_pit_8254.py -v`

cocotb-test - **Framework**: pytest integration for CocoTB - **Purpose**: Bridge between pytest and CocoTB simulation - **Documentation**: https://github.com/themperek/cocotb-test

6.1.6 Design Patterns and Standards

Reset Macro Standards - **File**: `rtl/amba/includes/reset_defs.svh` - **Description**: Repository-standard reset handling macros - **Macros**: `ALWAYS_FF_RST`, `RST_ASSERTED` - **Rationale**: FPGA-friendly reset inference, consistent reset polarity

FPGA Synthesis Attributes - **Reference**: Xilinx Vivado Synthesis Guide, Quartus Synthesis Handbook - **Attributes**: `ram_style`, `use_dsp`, synthesis directives - **Purpose**: Guide FPGA synthesis for optimal resource usage

Testbench Architecture - **Pattern**: Separate TB classes from test runners - **Location**: `dv/tbclasses/{block}/` for TB classes, `dv/tests/{block}/` for runners - **Rationale**: Reusability, maintainability, composition

6.1.7 External Resources

8254 Historical Context - **Platform**: IBM PC/AT and compatibles - **Usage**: System timer, speaker control, PC architecture timer - **Legacy**: Widely emulated in virtualization and retro computing

APB Timer Implementations - Various vendor implementations of APB-attached timers - ARM Cortex-M system timer examples - Open-source APB peripheral repositories

6.1.8 Version Control and Issue Tracking

Repository - **URL**: https://github.com/sean-galloway/rtldesignsherpa (if public) - **Branch**: main - **Component Path**: `projects/components/legacy-ip/retro_legacy_blocks/`

Related Issues - Implementation tracked in `vault/Tasks/RLB/` (per-block sub-areas as needed) - Status, stated deviations and the known test-vs-design gap documented in the RTL README.md

6.1.9 Change History

This specification is version-controlled with the RTL implementation. See git history for detailed change tracking:

```
# View PIT 8254 commit history
git log --follow --
projects/components/legacy-ip/retro_legacy_blocks/rtl/pit_8254/
git log --follow --
projects/components/legacy-ip/retro_legacy_blocks/docs/pit_8254_mas/
```

6.1.10 Related Components

APB HPET (High Precision Event Timer) - **Status**: Production ready - **Similarity**: APB timer peripheral, similar architecture - **Differences**: 64-bit counters, multiple modes, periodic operation

Future RLB Components - **8259 PIC**: Interrupt controller - **RTC**: Real-time clock - **SMBus**: System management bus - See `PRD.md` for complete roadmap

---

Document Revision - **Version**: 1.0 - **Last Updated**: 2025-11-08 - **Maintained By**: RTL Design Sherpa Project - **Documentation Support**: Claude

---

For More Information: - Check the RTL `README.md` for latest implementation details - Review `README.md` for integration examples - Consult `PRD.md` for requirements and roadmap

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

7 APB PIT 8254 - Block Hierarchy

7.1 Overview

The APB PIT 8254 follows a clean three-layer architecture for maintainability and clarity:

```
apb4_pit_8254 (Top Level)
├── apb4_slave or apb4_slave_cdc (APB Interface)
│   └── Protocol conversion: APB → cmd/rsp interface
│
├── pit_config_regs (Configuration Registers)
│   ├── peakrdl_to_cmdrsp (Protocol Adapter)
│   │   └── Converts cmd/rsp → PeakRDL cpuif_apb protocol
│   │
│   └── pit_regs (PeakRDL Generated Register File)
│       └── Register storage with hwif interface
│
└── pit_core (Core Logic)
    ├── pit_counter (Counter 0)
    ├── pit_counter (Counter 1)
    └── pit_counter (Counter 2)
```

7.2 Ports

7.2.1 Interface Summary

Between Blocks:

From Block	To Block	Interface	Signals
apb4_pit_8254	apb4_slave	APB4	s_apb_PSEL, s_apb_PENABLE, s_apb_PWRITE, s_apb_PADDR, s_apb_PWDATA, s_apb_PSTRB, s_apb_PPROT, s_apb_PRDATA, s_apb_PREADY, s_apb_PSLVERR
apb4_slave	pit_config_regs	cmd/rsp	cmd_valid/

From Block	To Block	Interface	Signals
			ready, cmd_pwrite, cmd_paddr, cmd_pwdata, cmd_pstrb; rsp_valid/ready, rsp_prdata, rsp_pslverr
pit_config_regs	pit_regs	cpuif_apb	Various PeakRDL interface signals
pit_regs	pit_config_regs	hwif	hwif_out, hwif_in (struct interfaces)
pit_config_regs	pit_core	Control	pit_enable, control_word, control_wr, counter_data, counter_wr, counter_rd
pit_core	pit_counter	Per-Counter	count_reg_in/ wr/rd, mode, rw_mode, bcd, control_wr, latch_cmd, gate (synchronized), clk_en, out, status

External Interfaces:

Interface	Direction	Purpose
APB4	Bidirectional	Register access from CPU
GATE[2:0]	Input	External counter

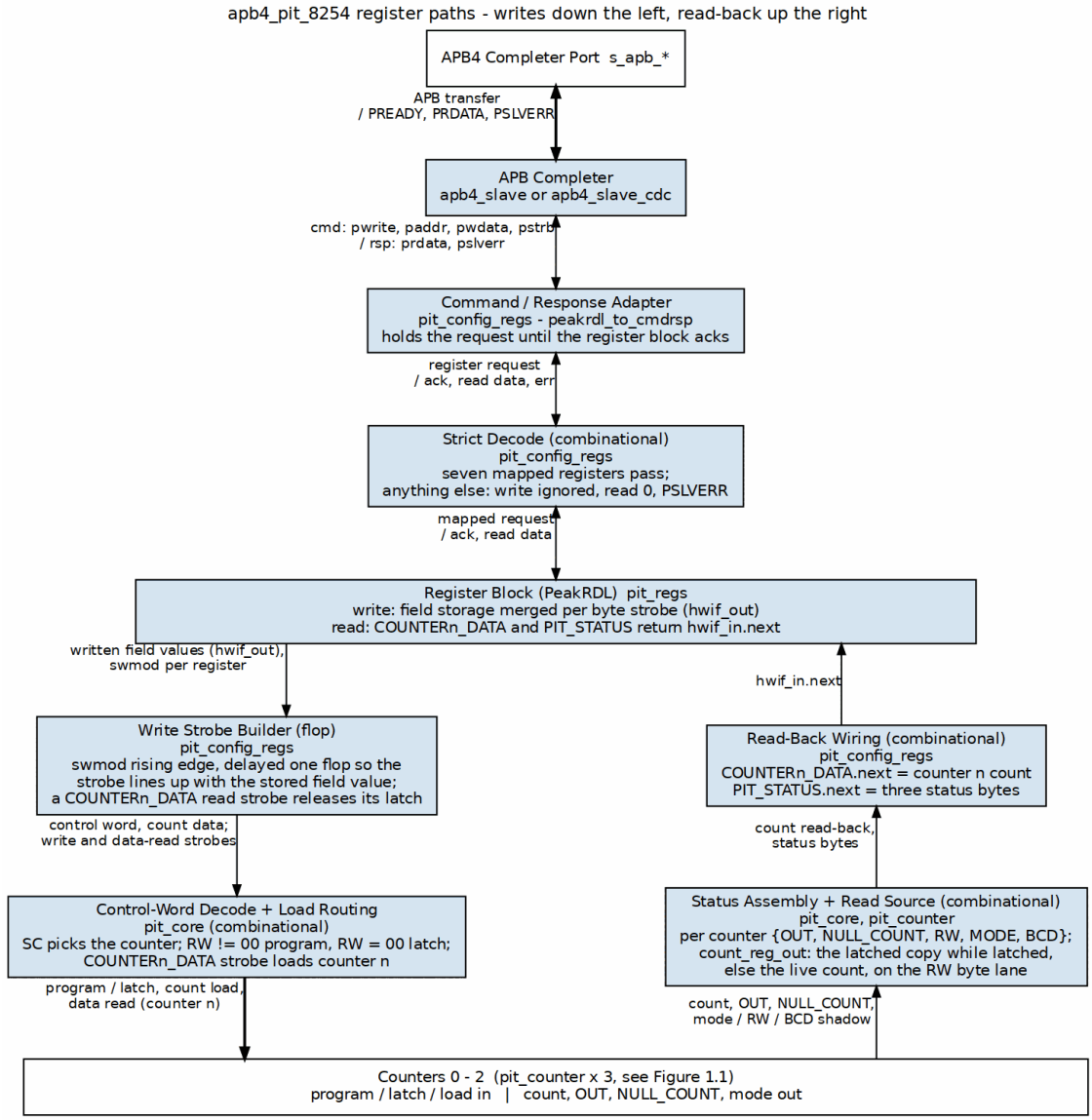
Interface	Direction	Purpose
		enable controls
timer_irq[2:0]	Output	Interrupt outputs (driven by OUT signals)
pit_clk	Input	Timer clock (when CDC_ENABLE=1)

7.3 Functional Description

7.3.1 Dataflow Between Blocks

Write Path (Software -> Hardware) runs down the left of Figure 2.1, and the **Read Path (Hardware -> Software)** climbs the right.

7.3.2 Figure 2.1: APB PIT 8254 Register Write and Read Paths



APB PIT 8254 Register Write and Read Paths

7.3.3 Block Responsibilities

apb4_pit_8254 (Top Level) - Module instantiation and parameter propagation - Signal routing between major blocks - Optional clock domain crossing selection - Top-level I/O connection

apb4_slave / apb4_slave_cdc (APB Interface) - APB protocol state machine and transaction control (the address decode lives in pit_config_regs) - Optional CDC when `CDC_ENABLE=1` - Carries PSLVERR back to the bus for any address pit_config_regs drops (see ch03)

pit_config_regs (Configuration Registers) - PeakRDL register file instantiation - Protocol adaptation (cmd/rsp ↔️ cpuif_apb) - Strict address decode: the seven mapped registers pass through, every other address in the 4 KB window is acknowledged locally as a dropped access (write ignored, read 0, PSLVERR) - One-cycle command strobes, edge-detected from the adapter's held request and delayed one flop so they line up with the stored field value – one write, one load, byte strobes already merged - Counter readback connection, plus a data-read strobe per counter that releases the latch - Control-word and counter-load write strobes (the counter-select decode of the control word itself is in pit_core.sv)

pit_regs (PeakRDL Generated) - Register storage (flip-flops) - Hardware interface (hwif_out, hwif_in) - Reset value initialization - Access control (RO, WO, RW fields)

pit_core (Core Logic) - Counter instance management (3 counters) - Control word routing to selected counter, split into a program strobe (RW != 00) and a latch command (RW = 00) - GATE input synchronizer, SYNC_STAGES flops per counter, unconditional in both clocking configurations - Data routing to/from selected counter - Global enable (clock enable) distribution - Status aggregation from all counters

pit_counter (Individual Counter) - 16-bit down-counter logic, binary or BCD; count 0 means 65536 - Mode 0 counting – two flags and the count, no state machine - GATE pause/resume (GATE sits in the decrement condition) - Counter latch: frozen copy plus a latched flag, released by the data read - OUT signal generation, one steady state after terminal count - Control word storage (mode, RW mode, BCD flag) - Count load on the RW-selected byte lane

7.3.4 Signal Flow Examples

Example 1: Writing Counter 0 Control Word

```
CPU writes 0x30 to PIT_CONTROL (0x004)
    ↓
APB transaction on paddr=0x004, pwdata=0x30, pwrite=1
    ↓
apb4_slave asserts cmd_valid with cmd_pwrite=1, cmd_paddr=0x004,
cmd_pwdata=0x30
    ↓
peakrdl_to_cmdrsp converts to cpuif_apb protocol
    ↓
pit_regs.PIT_CONTROL field updates (hwif_out)
    ↓
pit_config_regs fires the one-cycle PIT_CONTROL strobe, aligned to the
stored field
    ↓
pit_core decodes: SC=00 (Counter 0), RW=11 (a program, not a latch),
MODE=000, BCD=0
    ↓
Asserts cfg_control_wr on counter 0 (RW=00 would assert cfg_latch_cmd
instead)
    ↓
```

```
pit_counter[0] shadows mode/RW/BCD, sets NULL_COUNT, drives OUT low,
aborts any count in progress
```

Example 2: Reading Counter 1 Value

```
CPU reads from COUNTER1_DATA (0x014)
    ↓
APB read transaction on paddr=0x014, pwrite=0
    ↓
apb4_slave asserts cmd_valid with cmd_pwrite=0, cmd_paddr=0x014 (read)
    ↓
peakrdl_to_cmdrsp converts to cpuif_apb read
    ↓
pit_regs needs COUNTER1_DATA.counter1_data field value
    ↓
hwif_in.COUNTER1_DATA.counter1_data.next = counter1_readback
    ↓ (continuously connected)
pit_core routes count_reg_out from counter 1 instance
    ↓
pit_counter[1].count_reg_out reflects current r_count value
    ↓
Value propagates back through read path
    ↓
prdata returns current counter 1 value to CPU
```

Example 3: Counter Decrement Operation

```
pit_counter instance in COUNTING state
    ↓
i_clk_en=1 (PIT enabled), i_gate=1 (synchronized GATE high)
    ↓
w_tick = r_counting && i_clk_en && i_gate   // GATE low: no tick,
count holds
    ↓
On rising edge of clk:
    if (w_tick) begin
        r_count <= w_next_count;             // decremented value
(binary or BCD)
        if (w_next_count == 16'd0) begin     // terminal count, tested
AFTER the decrement
            r_out      <= 1'b1;              //   so a load of 0
counts 65536
            r_counting <= 1'b0;              //   and nothing re-arms
it but a new load
        end
    end
    ↓
Updated r_count value available at count_reg_out
```

```
    ↓
Updated r_out value propagates to OUT signal and timer_irq output
```

7.4 Timing

7.4.1 Clock Domain Considerations

Single Clock Configuration (CDC_ENABLE=0): - All blocks use `pclk` - No domain crossing required - Direct connections throughout hierarchy - Lowest latency (2-3 cycle register access)

Dual Clock Configuration (CDC_ENABLE=1): - `apb4_slave_cdc` uses `pclk` for APB interface - `pit_config_regs` and `pit_core` use `pit_clk` - CDC logic inside `apb4_slave_cdc` handles crossing - Higher latency (4-6 cycle register access) - Independent timer clock frequency

7.5 Design Notes

7.5.1 Reset Behavior Flow

Power-On Reset (presetn asserted):

```
presetn = 0 (active-low reset asserted)
    ↓
All apb4_slave state machines → IDLE
    ↓
All pit_regs fields → reset values (0x00 for most)
    ↓
pit_core: pit_enable → 0 (disabled)
    ↓
All pit_counter instances:
    r_count → 16'd0
    r_null_count → 1'b1 (no count loaded)
    r_counting → 1'b0
    r_out → 1'b0
    r_mode → 3'b000
    r_rw_mode → 2'b00
    r_bcd → 1'b0
    ↓
External outputs:
    timer_irq[2:0] → 3'b000
    prdata → 32'h0
    pready → 1'b0
```

Reset Deassertion (presetn = 1):

```
presetn = 1 (reset released)
    ↓
All state machines begin normal operation
    ↓
```

```
Counters remain idle (r_null_count=1) until programmed
    ↓
Ready to accept APB transactions
```

Version: 1.0 **Last Updated:** 2025-11-08

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

8 APB PIT 8254 - Top-Level Interface

8.1 Overview

`apb4_pit_8254` is the top-level module: an APB4 slave on one side, GATE inputs and interrupt outputs on the other, and a parameter-selected clocking scheme in the middle. This chapter is the integration contract – pin list, parameters, address decode, and the reset sequence your SoC has to honor.

8.2 Parameters

Parameter	Type	Default	Valid Values	Description
`NUM_COUNTERS`	int	3	Currently fixed at 3	Number of independent counters. Parameterized, but the current implementation only supports 3.
`CDC_ENABLE`	bit	0	0 = single clock, 1 = dual clock with CDC	Selects the clocking scheme. `CDC_ENABLE=0` uses `apb4_slave` and ignores `pit_clk`/`pit_resetn`; `CDC_ENABLE=1` uses `apb4_slave_cdc` and requires both.
`USE_JOHNSON`	int	0	0 = Gray-coded CDC FIFO pointers, 1 = Johnson-coded	Forwarded to the CDC block's async FIFOs. Gray requires a power-of-2 depth; Johnson allows any depth. Only meaningful when

Parameter	Type	Default	Valid Values	Description
				CDC_ENABLE=1.
SYNC_STAGES	int	2	>= 2	Depth of the gate_in synchronizer in pit_core. Present in both clocking configurations – the pin is asynchronous to the counting clock either way. A GATE transition reaches the counter SYNC_STAGES counting clocks after it happens.

8.3 Ports

8.3.1 Module Declaration

```
module apb4_pit_8254 #(
    parameter int NUM_COUNTERS = 3,  // Number of counters (fixed at
3)
    parameter bit CDC_ENABLE   = 0,  // 0=single clock, 1=dual clock
with CDC
    parameter int USE_JOHNSON  = 0,  // CDC FIFO pointer encoding:
0=Gray, 1=Johnson
    parameter int SYNC_STAGES  = 2   // gate_in synchronizer depth, >=
2
) (
    // Clock and Reset - Dual Domain
    input  wire                     pclk,        // APB clock domain
    input  wire                     presetn,     // APB reset (active
low)
    input  wire                     pit_clk,     // PIT clock domain
(CDC_ENABLE=1)
    input  wire                     pit_resetn,  // PIT reset (active
low)
```

```
    // APB4 Slave Interface
    input  wire                     s_apb_PSEL,
    input  wire                     s_apb_PENABLE,
    output wire                     s_apb_PREADY,
    input  wire [11:0]              s_apb_PADDR,
    input  wire                     s_apb_PWRITE,
    input  wire [31:0]              s_apb_PWDATA,
    input  wire [3:0]               s_apb_PSTRB,
    input  wire [2:0]               s_apb_PPROT,
    output wire [31:0]              s_apb_PRDATA,
    output wire                     s_apb_PSLVERR,

    // Timer Interface
    input  wire [NUM_COUNTERS-1:0] gate_in,     // GATE inputs for
counters
    output wire [NUM_COUNTERS-1:0] timer_irq    // Interrupt outputs
);
```

8.3.2 Signal Groups

APB Clock and Reset: | Signal | Direction | Width | Description | |———|————|———-| ————-| | `pclk` | Input | 1 | APB bus clock. All APB signals are synchronous to this clock. | | `presetn` | Input | 1 | APB reset, active-low. Asynchronous assertion, synchronous deassertion (generated register file is the exception – it resets synchronously; see ch01 clocks). |

APB Interface Signals: | Signal | Direction | Width | Description | |———|————|———-| ————-| | `s_apb_PADDR` | Input | 12 | APB address. All 12 bits are compared against the seven mapped registers; anything else is dropped with PSLVERR. See Address Map below. | | `s_apb_PSEL` | Input | 1 | APB select. Asserted by interconnect when this peripheral is accessed. | | `s_apb_PENABLE` | Input | 1 | APB enable. Asserted in second cycle of transfer (access phase). | | `s_apb_PWRITE` | Input | 1 | APB write/read. 1=write, 0=read. | | `s_apb_PWDATA` | Input | 32 | APB write data. Valid only when `s_apb_PWRITE=1`. | | `s_apb_PSTRB` | Input | 4 | APB write strobes (byte lane enables). Honoured all the way through: the register block merges the strobed bytes into the stored value, and a counter load takes that merged value, so a PSTRB=0x1 write to COUNTERx_DATA loads {stored high byte, new low byte}. A write that strobes none of a register's bytes changes nothing and fires no command. | | `s_apb_PPROT` | Input | 3 | APB protection attributes. Accepted for protocol completeness; not used by the decode. | | `s_apb_PRDATA` | Output | 32 | APB read data. Valid when `s_apb_PREADY=1` and `s_apb_PWRITE=0`. | | `s_apb_PREADY` | Output | 1 | APB ready. The wrapper converts APB to an internal command/response handshake, so PREADY inserts wait states: typically 2-3 `pclk` cycles single-clock, 4-6 cycles across the CDC. | | `s_apb_PSLVERR` | Output | 1 | APB slave error. Asserted for any access whose address is not one of the seven mapped registers: the write is ignored, the read returns 0, and the transfer completes with PSLVERR high. Mapped accesses never error. |

PIT Clock and Reset: | Signal | Direction | Width | Description | |———|————|———-| ————-| | `pit_clk` | Input | 1 | Timer clock. Used when `CDC_ENABLE=1` for independent timer

clock domain. Ignored when CDC_ENABLE=0. | | pit_resetn | Input | 1 | Timer reset, active-low. Used when CDC_ENABLE=1. Should be synchronous to pit_clk. Ignored when CDC_ENABLE=0. |

Counter Control and Status: | Signal | Direction | Width | Description | |———|————|———-| ————-| | gate_in[2:0] | Input | 3 | GATE inputs for counters 0, 1, 2, active high. In Mode 0 a low GATE pauses the count where it stands and a high GATE resumes it from that value – no reload, no restart – as on the 8254. A load is unaffected by GATE. The pins are treated as asynchronous in both clocking configurations and pass a SYNC_STAGES-flop synchronizer (default 2) on the counting clock, so a transition takes effect SYNC_STAGES counting clocks after the pin moves. No external synchronizer is needed. | | timer_irq[2:0] | Output | 3 | Timer interrupt outputs. Driven by OUT signals from counters 0, 1, 2. High when terminal count reached (Mode 0). |

8.4 Functional Description

8.4.1 Address Map

s_apb_PADDR is 12 bits and all 12 are decoded: only the seven mapped registers below are software-visible. Every other address in the 4 KB window – 0x01C, 0x020 and up – is dropped: the write is ignored, the read returns 0, and the access completes with PSLVERR. There are no aliases, and the command strobes (control-word execution, counter loads, latch release) are reached only through an address the decode has already accepted, so a write that stores is a write that executes. This is the same policy as the ioapic and pic_8259 blocks.

Address Range	Register	Access	Description
0x000	PIT_CONFIG	RW	Global configuration (enable)
0x004	PIT_CONTROL	WO	Control word (8254-compatible)
0x008	PIT_STATUS	RO	Status readback (3 counters)
0x00C	RESERVED	-	Reserved
0x010	COUNTER0_DATA	RW	Counter 0 value
0x014	COUNTER1_DATA	RW	Counter 1 value
0x018	COUNTER2_DATA	RW	Counter 2 value
0x01C	-	-	Unmapped: dropped with PSLVERR (reads

Address Range	Register	Access	Description
			0, write ignored)
0x020-0xFFF	-	-	Unmapped: dropped with PSLVERR (reads 0, write ignored)

Integration Note: When integrating into a larger address space, these addresses are relative to the base address assigned to the PIT. In the shipped RLB the PIT occupies window 2, so its base is `0xFEC0_2000` and PIT_CONFIG is at absolute address `0xFEC0_2000`.

8.4.2 Error Response

Any access to an address other than the seven mapped registers is acknowledged locally by `pit_config_regs` and answered with `s_apb_PSLVERR` high: the write changes nothing, the read returns 0. The dropped access still completes – the adapter holds its request until an acknowledge, so the decode acks the drop itself rather than leaving the bus waiting. Mapped accesses never raise PSLVERR. Software can rely on the bus error to catch a bad pointer into this window.

8.5 Timing

8.5.1 Clock Domain Configuration

Single Clock Mode (CDC_ENABLE=0):

Connections:

```
// All logic uses pclk
apb4_slave      uses: pclk, presetn
pit_config_regs uses: pclk, presetn
pit_core       uses: pclk, presetn
pit_counter[*] uses: pclk, presetn

// pit_clk and pit_resetn are not used
```

Use Cases: - Timer clock same as APB bus clock - Simplified integration - Lower latency (no CDC overhead) - FPGA implementations with single clock domain

Dual Clock Mode (CDC_ENABLE=1):

Connections:

```
// APB interface uses pclk
apb4_slave_cdc uses: pclk for APB side, pit_clk for timer side
                     presetn for APB reset, pit_resetn for timer reset

// Timer logic uses pit_clk
pit_config_regs uses: pit_clk, pit_resetn
```

```
pit_core       uses: pit_clk, pit_resetn
pit_counter[*] uses: pit_clk, pit_resetn
```

Use Cases: - Timer requires independent clock frequency - Timer clock faster/slower than APB bus - Power optimization (gate APB clock, keep timer running) - Multiple clock domain systems

CDC Considerations: - APB transactions take 4-6 `pit_clk` cycles (vs 2-3 in single-clock mode) - Both clocks must be free-running during transactions - Ensure proper reset sequencing (both domains reset before use) - `gate_in` needs no special handling: its synchronizer runs on the counting clock in both configurations (`SYNC_STAGES` flops, default 2)

8.5.2 Reset Requirements

Power-On Reset Sequence:

1. Assert both resets:

   ```
   presetn = 0
   pit_resetn = 0  (if CDC_ENABLE=1)
   ```

2. Hold for minimum 10 clock cycles (of slowest clock):

   ```
   wait >= 10 * max(pclk_period, pit_clk_period)
   ```

3. Deassert resets synchronously:

   ```
   // On rising edge of pclk
   presetn = 1

   // On rising edge of pit_clk (if CDC_ENABLE=1)
   pit_resetn = 1
   ```

4. Wait for reset propagation:

   ```
   wait >= 5 * max(pclk_period, pit_clk_period)
   ```

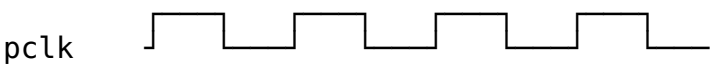
5. PIT now ready for register access

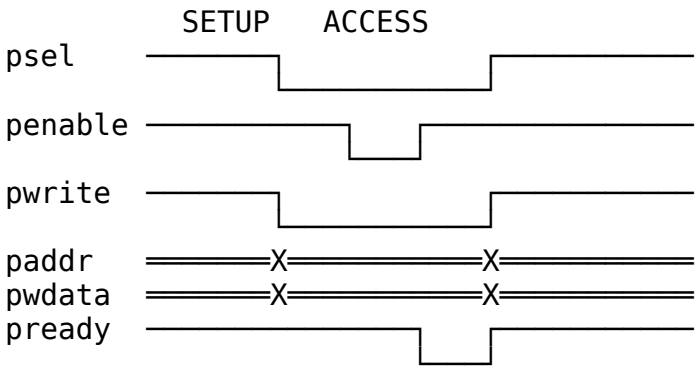
Reset During Operation:

If resetting during operation: - Disable PIT first: `write(PIT_CONFIG, 0x00)` - Wait for counters to stop: `wait >= 2 * pit_clk_period` - Assert reset - Follow power-on reset sequence from step 2

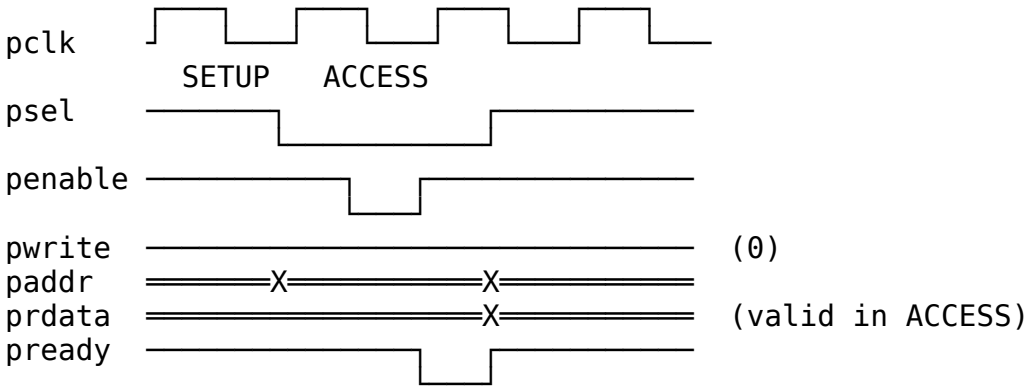
8.5.3 APB Protocol Timing

Write Transaction:





Read Transaction:



8.6 Usage Example

8.6.1 Single Clock Integration

```
apb4_pit_8254 #(
    .NUM_COUNTERS(3),
    .CDC_ENABLE(0)
) u_apb4_pit_8254 (
    .pclk                   (pclk),
    .presetn                (presetn),
    .pit_clk                (pit_clk),
    .pit_resetn             (pit_resetn),
    .s_apb_PSEL             (s_apb_PSEL),
    .s_apb_PENABLE          (s_apb_PENABLE),
    .s_apb_PREADY           (s_apb_PREADY),
    .s_apb_PADDR            (s_apb_PADDR),
    .s_apb_PWRITE           (s_apb_PWRITE),
    .s_apb_PWDATA           (s_apb_PWDATA),
    .s_apb_PSTRB            (s_apb_PSTRB),
    .s_apb_PPROT            (s_apb_PPROT),
    .s_apb_PRDATA           (s_apb_PRDATA),
    .s_apb_PSLVERR          (s_apb_PSLVERR),
```

```
    .gate_in                (gate_in),
    .timer_irq              (timer_irq)
);
```

8.6.2 Dual Clock Integration

```
apb4_pit_8254 #(
    .NUM_COUNTERS(3),
    .CDC_ENABLE(1)
) u_apb4_pit_8254 (
    .pclk                   (pclk),
    .presetn                (presetn),
    .pit_clk                (pit_clk),
    .pit_resetn             (pit_resetn),
    .s_apb_PSEL             (s_apb_PSEL),
    .s_apb_PENABLE          (s_apb_PENABLE),
    .s_apb_PREADY           (s_apb_PREADY),
    .s_apb_PADDR            (s_apb_PADDR),
    .s_apb_PWRITE           (s_apb_PWRITE),
    .s_apb_PWDATA           (s_apb_PWDATA),
    .s_apb_PSTRB            (s_apb_PSTRB),
    .s_apb_PPROT            (s_apb_PPROT),
    .s_apb_PRDATA           (s_apb_PRDATA),
    .s_apb_PSLVERR          (s_apb_PSLVERR),
    .gate_in                (gate_in),
    .timer_irq              (timer_irq)
);
```

Version: 1.1 **Last Updated:** 2026-09-09

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

9 APB PIT 8254 - Initialization and Programming Guide

Bring-up is seven steps, and the order matters. Disable first, program everything, verify, then enable. The full sequence is below, followed by a complete function you can lift as-is.

9.1 Power-On Initialization Sequence

Step 1: Verify Reset State

After power-on or reset, verify the PIT is in expected reset state:

```
// Expected reset values
assert(read_register(PIT_CONFIG) == 0x00000000);    // PIT disabled
assert(read_register(PIT_STATUS) == 0x00404040);    // All counters:
NULL_COUNT=1 (bit 6 of each status byte), OUT=0
assert(read_register(COUNTER0_DATA) == 0x00000000); // No count loaded
assert(read_register(COUNTER1_DATA) == 0x00000000);
assert(read_register(COUNTER2_DATA) == 0x00000000);
```

Step 2: Configure Global Settings

The PIT must be disabled during initial configuration:

```
// Ensure PIT is disabled before programming
write_register(PIT_CONFIG, 0x00);  // PIT_ENABLE=0
```

Step 3: Program Counter Control Words

Each counter must be configured with a control word before use:

```
// Configure Counter 0: Mode 0, binary, LSB+MSB access
uint32_t control_word_0 = (0 << 6) |  // SC=00 (Counter 0)
                          (3 << 4) |  // RW=11 (LSB+MSB)
                          (0 << 1) |  // MODE=000 (Mode 0)
                          (0 << 0);   // BCD=0 (binary)
write_register(PIT_CONTROL, control_word_0);  // Write 0x30

// Configure Counter 1: Mode 0, binary, LSB+MSB access
uint32_t control_word_1 = (1 << 6) |  // SC=01 (Counter 1)
                          (3 << 4) |  // RW=11 (LSB+MSB)
                          (0 << 1) |  // MODE=000 (Mode 0)
                          (0 << 0);   // BCD=0 (binary)
write_register(PIT_CONTROL, control_word_1);  // Write 0x70

// Configure Counter 2: Mode 0, binary, LSB+MSB access
uint32_t control_word_2 = (2 << 6) |  // SC=10 (Counter 2)
                          (3 << 4) |  // RW=11 (LSB+MSB)
                          (0 << 1) |  // MODE=000 (Mode 0)
```

```
                          (0 << 0);   // BCD=0 (binary)
write_register(PIT_CONTROL, control_word_2);  // Write 0xB0
```

Step 4: Load Initial Count Values

After configuring control words, load count values:

```
// Load Counter 0 with count of 1000
write_register(COUNTER0_DATA, 1000);

// Load Counter 1 with count of 2000
write_register(COUNTER1_DATA, 2000);

// Load Counter 2 with count of 5000
write_register(COUNTER2_DATA, 5000);
```

Step 5: Verify Configuration

Read back status to confirm configuration:

```
uint32_t status = read_register(PIT_STATUS);

// Extract counter 0 status
uint8_t counter0_status = status & 0xFF;
assert((counter0_status & 0x40) == 0);  // NULL_COUNT should be 0
(count loaded)
assert((counter0_status & 0x30) == 0x30);  // RW_MODE should be 3
(LSB+MSB)
assert((counter0_status & 0x0E) == 0x00);  // MODE should be 0
assert((counter0_status & 0x01) == 0x00);  // BCD should be 0
```

Step 6: Enable PIT and Start Counting

```
// Enable global PIT operation
write_register(PIT_CONFIG, 0x01);  // PIT_ENABLE=1

// Counters will now begin counting (assuming GATE inputs are high)
```

Step 7: Monitor Operation

```
// Wait for counter 0 to reach terminal count
while (1) {
    uint32_t status = read_register(PIT_STATUS);
    uint8_t counter0_status = status & 0xFF;
    bool out_high = (counter0_status >> 7) & 0x1;

    if (out_high) {
        printf("Counter 0 reached terminal count!\n");
        break;
```

```
    }

    // Optional: read current counter value
    uint32_t current_count = read_register(COUNTER0_DATA) & 0xFFFF;
    printf("Counter 0 current value: %u\n", current_count);
}
```

9.2 Complete Initialization Function

```
/**
 * Initialize PIT with three counters
 * @param counter0_count Initial count for counter 0
 * @param counter1_count Initial count for counter 1
 * @param counter2_count Initial count for counter 2
 * @return 0 on success, -1 on error
 */
int pit_initialize(uint16_t counter0_count, uint16_t counter1_count,
uint16_t counter2_count) {
    // Step 1: Disable PIT during configuration
    write_register(PIT_CONFIG, 0x00);

    // Step 2: Configure counter 0 control word
    // Mode 0, binary, LSB+MSB access
    write_register(PIT_CONTROL, 0x30);

    // Step 3: Load counter 0 count value
    write_register(COUNTER0_DATA, counter0_count);

    // Step 4: Configure counter 1 control word
    write_register(PIT_CONTROL, 0x70);

    // Step 5: Load counter 1 count value
    write_register(COUNTER1_DATA, counter1_count);

    // Step 6: Configure counter 2 control word
    write_register(PIT_CONTROL, 0xB0);

    // Step 7: Load counter 2 count value
    write_register(COUNTER2_DATA, counter2_count);

    // Step 8: Verify configuration
    uint32_t status = read_register(PIT_STATUS);

    // Check all counters have counts loaded (NULL_COUNT=0)
    if ((status & 0x00404040) != 0) {
        printf("ERROR: Counter configuration failed (NULL_COUNT bits
```

```
set)\n");
        return -1;
    }

    // Step 9: Enable PIT
    write_register(PIT_CONFIG, 0x01);

    printf("PIT initialized successfully\n");
    printf("  Counter 0: %u counts\n", counter0_count);
    printf("  Counter 1: %u counts\n", counter1_count);
    printf("  Counter 2: %u counts\n", counter2_count);

    return 0;
}
```

9.3 Usage Example

```
int main(void) {
    // Initialize PIT with different count values
    if (pit_initialize(1000, 5000, 10000) != 0) {
        printf("PIT initialization failed!\n");
        return -1;
    }

    // Monitor counter 0 terminal count
    printf("Waiting for Counter 0 to reach terminal count...\n");

    while (1) {
        uint32_t status = read_register(PIT_STATUS);
        if (status & 0x80) {  // Counter 0 OUT bit
            printf("Counter 0 reached terminal count!\n");
            break;
        }

        // Print current count every 100 iterations
        static int count = 0;
        if (++count >= 100) {
            uint32_t current = read_register(COUNTER0_DATA) & 0xFFFF;
            printf("Counter 0: %u\n", current);
            count = 0;
        }
    }

    // Disable PIT after use
    write_register(PIT_CONFIG, 0x00);
```

```
    return 0;
}
```

9.4 Runtime Configuration Changes

Changing Count Value During Operation:

```
// To change counter 0 count value while running:

// 1. Disable PIT (stops all counters)
write_register(PIT_CONFIG, 0x00);

// 2. Write new count value
write_register(COUNTER0_DATA, new_count);

// 3. Re-enable PIT (counter restarts with new value)
write_register(PIT_CONFIG, 0x01);
```

Alternative: Write While Enabled (Immediate Restart)

```
// Writing counter data while enabled causes immediate reload and
restart
// No need to disable PIT first (but counter will restart from new
value)

write_register(COUNTER0_DATA, new_count);  // Counter reloads and
restarts
```

9.5 Common Initialization Errors

Error 1: Enabling PIT Before Programming

```
// WRONG: Enable before configuration
write_register(PIT_CONFIG, 0x01);  // Enable too early!
write_register(PIT_CONTROL, 0x30);
write_register(COUNTER0_DATA, 1000);

// CORRECT: Configure first, then enable
write_register(PIT_CONFIG, 0x00);  // Disable during config
write_register(PIT_CONTROL, 0x30);
write_register(COUNTER0_DATA, 1000);
write_register(PIT_CONFIG, 0x01);  // Enable after config
```

Error 2 (retracted): “Not Waiting for Count Load” is not a real hazard

```
// FINE: the count load is not gated by PIT_ENABLE, back-to-back APB
// writes are serialized, and counting proceeds whenever PIT_ENABLE
and
```

```
// GATE are both high
write_register(COUNTER0_DATA, 1000);
write_register(PIT_CONFIG, 0x01);

// If you want confirmation anyway, NULL_COUNT (status bit 6) clears
once
// the counter has taken the load:
uint32_t status = read_register(PIT_STATUS);
assert((status & 0x40) == 0);
```

Error 3: Wrong Control Word Format

```
// WRONG: Incorrect bit positions
uint32_t cw = (0 << 0) |  // Counter select in wrong position!
              (3 << 6) |  // RW in wrong position!
              (0 << 4);   // Mode in wrong position!

// CORRECT: Proper bit positions per 8254 spec
uint32_t cw = (0 << 6) |  // SC[7:6] = Counter select
              (3 << 4) |  // RW[5:4] = Read/Write mode
              (0 << 1) |  // M[3:1] = Mode
              (0 << 0);   // BCD[0] = Counting mode
```

9.6 Debugging Initialization Issues

Check 1: Verify Register Access

```
// Test read/write capability
write_register(PIT_CONFIG, 0x02);  // Write non-zero value
uint32_t readback = read_register(PIT_CONFIG);
if (readback != 0x02) {
    printf("ERROR: Register access failed! Read 0x%08X, expected 0x02\
n", readback);
}
```

Check 2: Verify Control Word Decode

```
// After writing control word, read status to verify decode
write_register(PIT_CONTROL, 0x30);  // Counter 0, RW=11, Mode 0
uint32_t status = read_register(PIT_STATUS);
uint8_t counter0_status = status & 0xFF;

printf("Counter 0 Status: 0x%02X\n", counter0_status);
printf("  RW_MODE: %u (expect 3)\n", (counter0_status >> 4) & 0x3);
printf("  MODE: %u (expect 0)\n", (counter0_status >> 1) & 0x7);
printf("  BCD: %u (expect 0)\n", counter0_status & 0x1);
```

Check 3: Verify Count Load

```c
// After writing count, read back to verify
write_register(COUNTER0_DATA, 1234);
uint32_t readback = read_register(COUNTER0_DATA) & 0xFFFF;
if (readback != 1234) {
    printf("ERROR: Count value mismatch! Read %u, expected 1234\n",
readback);
}
```

Version: 1.0 **Last Updated:** 2025-11-08

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

10 APB PIT 8254 - Common Use Cases

Five recipes cover most of what people build with this block. Each one is self-contained – lift the code, adjust the clock frequency, and go.

10.1 Use Case 1: Simple Timeout Timer

Application: Generate an interrupt after a fixed delay

Implementation:

```
/**
 * Configure PIT counter as simple timeout timer
 * @param counter_num Counter to use (0, 1, or 2)
 * @param timeout_us Timeout in microseconds
 * @param clock_freq_hz PIT clock frequency in Hz
 * @return 0 on success, -1 on error
 */
int pit_timeout_start(uint8_t counter_num, uint32_t timeout_us,
uint32_t clock_freq_hz) {
    // Calculate count value from timeout
    // counts = (timeout_us * clock_freq_hz) / 1,000,000
    uint64_t counts = ((uint64_t)timeout_us * clock_freq_hz) /
1000000ULL;

    // Check if count fits in 16 bits
    if (counts > 65535) {
        printf("ERROR: Timeout too long (max %u us at %u Hz)\n",
               (uint32_t)((65535ULL * 1000000ULL) / clock_freq_hz),
clock_freq_hz);
        return -1;
    }

    // Disable PIT during configuration
    write_register(PIT_CONFIG, 0x00);

    // Build control word: Counter N, Mode 0, binary, LSB+MSB
    uint32_t control_word = (counter_num << 6) |  // SC = counter
number
                            (3 << 4) |            // RW = LSB+MSB
                            (0 << 1) |            // MODE = 0 (one-
shot)
                            (0 << 0);             // BCD = 0 (binary)
    write_register(PIT_CONTROL, control_word);

    // Load count value
    uint32_t data_reg_addr = 0x010 + (counter_num * 4);
```

```
    write_register(data_reg_addr, (uint32_t)counts);

    // Enable PIT
    write_register(PIT_CONFIG, 0x01);

    printf("Timeout timer started: %u us (%llu counts) on Counter %u\
n",
           timeout_us, counts, counter_num);

    return 0;
}

/**
 * Check if timeout has expired
 * @param counter_num Counter number (0, 1, or 2)
 * @return 1 if expired, 0 if still counting
 */
int pit_timeout_expired(uint8_t counter_num) {
    uint32_t status = read_register(PIT_STATUS);

    // Extract status byte for specified counter
    uint8_t counter_status = (status >> (counter_num * 8)) & 0xFF;

    // Check OUT bit (bit 7)
    return (counter_status >> 7) & 0x1;
}

/**
 * Usage example
 */
void example_timeout(void) {
    // Start 100ms timeout on Counter 0 (assuming 10 MHz PIT clock)
    pit_timeout_start(0, 100000, 10000000);

    // Poll until timeout expires
    printf("Waiting for timeout...\n");
    while (!pit_timeout_expired(0)) {
        // Do other work here
    }
    printf("Timeout expired!\n");

    // Disable PIT
    write_register(PIT_CONFIG, 0x00);
}
```

10.2 Use Case 2: RTOS System Tick

Application: Generate periodic interrupt for RTOS scheduler

Implementation:

```
/**
 * Configure PIT for RTOS system tick
 * @param tick_rate_hz Desired tick rate (e.g., 1000 for 1ms ticks)
 * @param clock_freq_hz PIT clock frequency
 * @return 0 on success, -1 on error
 */
// Tick period constant: the ISR must reload the ORIGINAL count.
Reading
// COUNTER0_DATA at terminal count returns the CURRENT value, which is
0 -
// reloading that would make the next "period" a single clock and
produce an
// interrupt storm.
static uint32_t pit_tick_counts;

int pit_rtos_tick_init(uint32_t tick_rate_hz, uint32_t clock_freq_hz)
{
    // Calculate counts per tick
    uint32_t counts = clock_freq_hz / tick_rate_hz;

    if (counts > 65535) {
        printf("ERROR: Tick rate too slow for 16-bit counter\n");
        return -1;
    }
    pit_tick_counts = counts;

    // Use Counter 0 for system tick
    write_register(PIT_CONFIG, 0x00);  // Disable

    // Configure Counter 0: Mode 0, binary, LSB+MSB
    write_register(PIT_CONTROL, 0x30);

    // Load count value
    write_register(COUNTER0_DATA, counts);

    // Enable PIT
    write_register(PIT_CONFIG, 0x01);

    printf("RTOS tick configured: %u Hz (%u counts per tick)\n",
           tick_rate_hz, counts);
```

```
    return 0;
}

/**
 * RTOS tick interrupt handler
 *
 * Note: PIT_STATUS reads have NO side effect - nothing here "clears"
the
 * interrupt by reading. OUT (and so timer_irq) stays high from
terminal
 * count until the reload below drives it low again; the reload IS the
 * acknowledge.
 */
void pit_tick_isr(void) {
    // Reload the ORIGINAL tick period (one-shot Mode 0 requires a
manual
    // reload after each terminal count; the reload also deasserts
OUT).
    write_register(COUNTER0_DATA, pit_tick_counts);

    // Call RTOS tick handler
    rtos_tick();
}

/**
 * Usage example with interrupt
 */
void example_rtos_tick(void) {
    // Initialize 1ms system tick (10 MHz clock)
    pit_rtos_tick_init(1000, 10000000);

    // Register interrupt handler
    register_interrupt_handler(TIMER0_IRQ, pit_tick_isr);

    // Enable timer interrupt
    enable_interrupt(TIMER0_IRQ);

    // RTOS runs, tick ISR called every 1ms
    while (1) {
        rtos_schedule();
    }
}
```

Note on Mode 0 for Periodic Operation:

Mode 0 is one-shot by design. For periodic operation, the interrupt handler must reload the counter after each terminal count. Future PIT implementations may add Mode 2 or Mode 3 for automatic periodic operation.

10.3 Use Case 3: Multiple Independent Timers

Application: Three different timeout values running concurrently

Implementation:

```
/**
 * Multi-timer manager structure
 */
typedef struct {
    uint8_t counter_num;
    uint32_t timeout_us;
    uint32_t counts;
    bool active;
} pit_timer_t;

static pit_timer_t timers[3];
static uint32_t pit_clock_freq = 10000000;  // 10 MHz

/**
 * Initialize multi-timer system
 */
void pit_multi_timer_init(void) {
    write_register(PIT_CONFIG, 0x00);  // Disable during init

    for (int i = 0; i < 3; i++) {
        timers[i].counter_num = i;
        timers[i].active = false;

        // Configure each counter: Mode 0, binary, LSB+MSB
        uint32_t control_word = (i << 6) | (3 << 4) | (0 << 1) | (0 <<
0);
        write_register(PIT_CONTROL, control_word);
    }

    write_register(PIT_CONFIG, 0x01);  // Enable
}

/**
 * Start a timer
 */
int pit_timer_start(uint8_t timer_id, uint32_t timeout_us) {
    if (timer_id >= 3) return -1;
```

```
    pit_timer_t *timer = &timers[timer_id];

    // Calculate counts
    uint64_t counts = ((uint64_t)timeout_us * pit_clock_freq) /
1000000ULL;
    if (counts > 65535) return -1;

    timer->timeout_us = timeout_us;
    timer->counts = (uint32_t)counts;
    timer->active = true;

    // Load count value
    uint32_t data_reg = 0x010 + (timer_id * 4);
    write_register(data_reg, timer->counts);

    printf("Timer %u started: %u us\n", timer_id, timeout_us);
    return 0;
}

/**
 * Check if timer expired
 */
bool pit_timer_expired(uint8_t timer_id) {
    if (timer_id >= 3 || !timers[timer_id].active) return false;

    uint32_t status = read_register(PIT_STATUS);
    uint8_t counter_status = (status >> (timer_id * 8)) & 0xFF;
    bool out = (counter_status >> 7) & 0x1;

    if (out) {
        timers[timer_id].active = false;
        printf("Timer %u expired\n", timer_id);
    }

    return out;
}

/**
 * Usage example
 */
void example_multi_timer(void) {
    pit_multi_timer_init();

    // Start three different timeouts
```

```
    pit_timer_start(0, 10000);   // 10 ms
    pit_timer_start(1, 50000);   // 50 ms
    pit_timer_start(2, 100000);  // 100 ms

    // Poll all timers
    while (1) {
        if (pit_timer_expired(0)) {
            printf("Task A: Timer 0 fired\n");
            // Handle timer 0 event
        }

        if (pit_timer_expired(1)) {
            printf("Task B: Timer 1 fired\n");
            // Handle timer 1 event
        }

        if (pit_timer_expired(2)) {
            printf("Task C: Timer 2 fired\n");
            // All done
            break;
        }
    }

    write_register(PIT_CONFIG, 0x00);  // Disable
}
```

10.4 Use Case 4: Performance Profiling

Application: Measure code execution time with microsecond precision

Implementation:

```
/**
 * Profiler using PIT counter
 */
typedef struct {
    uint32_t start_count;
    uint32_t clock_freq;
} pit_profiler_t;

static pit_profiler_t profiler;

/**
 * Initialize profiler (uses Counter 2)
 */
void pit_profiler_init(uint32_t clock_freq_hz) {
    profiler.clock_freq = clock_freq_hz;
```

```
    write_register(PIT_CONFIG, 0x00);

    // Configure Counter 2: Mode 0, binary, LSB+MSB
    write_register(PIT_CONTROL, 0xB0);

    // Load maximum count (65535) for longest measurement
    write_register(COUNTER2_DATA, 65535);

    write_register(PIT_CONFIG, 0x01);

    printf("Profiler initialized (max measurement: %u us)\n",
           (65535 * 1000000U) / clock_freq_hz);
}

/**
 * Start profiling measurement
 */
void pit_profile_start(void) {
    // Read current counter value as start reference
    profiler.start_count = read_register(COUNTER2_DATA) & 0xFFFF;
}

/**
 * Stop profiling and return elapsed time in microseconds
 */
uint32_t pit_profile_stop(void) {
    // Read current counter value
    uint32_t end_count = read_register(COUNTER2_DATA) & 0xFFFF;

    // Calculate elapsed counts (counter counts down)
    // In Mode 0 the counter STOPS at zero (it never wraps), so a
down-count
    // difference is always valid. If end_count is 0 with OUT high,
the
    // measured interval hit terminal count and the result is a floor,
not
    // an exact time - use a larger start count for longer intervals.
    uint32_t elapsed_counts = profiler.start_count - end_count;

    // Convert to microseconds
    uint32_t elapsed_us = (elapsed_counts * 1000000ULL) /
profiler.clock_freq;

    return elapsed_us;
}
```

```c
/**
 * Usage example
 */
void example_profiling(void) {
    pit_profiler_init(10000000);  // 10 MHz clock

    // Profile some code
    pit_profile_start();

    // Code to measure
    for (volatile int i = 0; i < 10000; i++) {
        // Busy loop
    }

    uint32_t elapsed = pit_profile_stop();
    printf("Code execution time: %u us\n", elapsed);

    // Profile another section
    pit_profile_start();
    complex_function();
    elapsed = pit_profile_stop();
    printf("Complex function: %u us\n", elapsed);

    write_register(PIT_CONFIG, 0x00);
}
```

10.5 Use Case 5: Watchdog Timer

Application: Detect system lockup

Implementation:

```c
/**
 * Watchdog timer using PIT Counter 1
 */
#define WATCHDOG_TIMEOUT_MS 1000  // 1 second timeout

static uint32_t watchdog_counts;  // shared with pit_watchdog_pet()

void pit_watchdog_init(uint32_t clock_freq_hz) {
    // Calculate counts for timeout
    uint32_t counts = (WATCHDOG_TIMEOUT_MS * clock_freq_hz) / 1000;
    watchdog_counts = counts;

    write_register(PIT_CONFIG, 0x00);
```

```
    // Configure Counter 1: Mode 0, binary, LSB+MSB
    write_register(PIT_CONTROL, 0x70);

    write_register(COUNTER1_DATA, counts);

    write_register(PIT_CONFIG, 0x01);

    printf("Watchdog initialized: %u ms timeout\n",
WATCHDOG_TIMEOUT_MS);
}

/**
 * Pet the watchdog (reset timer)
 */
void pit_watchdog_pet(void) {
    // Reload counter to reset timeout (same clock_freq_hz-derived
value the
    // init computed - never a hardcoded clock frequency)
    write_register(COUNTER1_DATA, watchdog_counts);
}

/**
 * Check if watchdog expired
 */
bool pit_watchdog_expired(void) {
    uint32_t status = read_register(PIT_STATUS);
    uint8_t counter1_status = (status >> 8) & 0xFF;
    return (counter1_status >> 7) & 0x1;
}

/**
 * Usage example
 */
void example_watchdog(void) {
    pit_watchdog_init(10000000);

    while (1) {
        // Main application loop
        do_work();

        // Pet watchdog to prevent timeout
        pit_watchdog_pet();

        // If do_work() hangs, watchdog will expire
        if (pit_watchdog_expired()) {
```

```
            printf("WATCHDOG TIMEOUT! System locked up!\n");
            system_reset();
        }
    }
}
```

10.6 Best Practices Summary

1. Always Disable PIT During Configuration

```
write_register(PIT_CONFIG, 0x00);  // Disable first
// Configure counters...
write_register(PIT_CONFIG, 0x01);  // Enable when done
```

2. Verify Count Values Fit in 16 Bits

```
if (counts > 65535) {
    printf("ERROR: Count too large\n");
    return -1;
}
```

3. Use Appropriate Counter for Application - Counter 0: System tick, main application timer - Counter 1: Watchdog, secondary timeout - Counter 2: Profiling, debugging measurements

4. No Wrap Handling Needed in Mode 0

```
// The counter STOPS at zero (never wraps), so the difference is
always valid
elapsed = start - end;
```

5. Clear Interrupts Properly

```
void pit_isr(void) {
    // Status reads have NO side effect; the counter reload below is
the
    // acknowledge (clears OUT / the irq)
    uint32_t status = read_register(PIT_STATUS);

    // Reload counter if needed (Mode 0)
    write_register(COUNTER0_DATA, reload_value);

    // Handle interrupt
    // ...
}
```

Version: 1.0 **Last Updated:** 2025-11-08

11 APB PIT 8254 - Register Map

11.1 Overview

Address	Name	Access	Description
0x000	PIT_CONFIG	RW	Global configuration
0x004	PIT_CONTROL	WO	Control word (8254-compatible)
0x008	PIT_STATUS	RO	Status readback (3 bytes)
0x00C	RESERVED	-	Reserved
0x010	COUNTER0_DATA	RW	Counter 0 value
0x014	COUNTER1_DATA	RW	Counter 1 value
0x018	COUNTER2_DATA	RW	Counter 2 value

These seven registers are the only software-visible addresses in the 4 KB window. Every other address – 0x01C, 0x020 and up – is dropped with PSLVERR: the write is ignored and the read returns 0. There are no aliases. See the top-level interface chapter for the decode policy.

11.2 PIT_CONFIG (0x000) - Global Configuration

Access: Read/Write **Reset Value:** 0x00000000

Bits	Name	Access	Reset	Description
[31:2]	RESERVED	RO	0	Reserved, read as 0
[1]	CLOCK_SELECT	RW	0	Clock source select. Storage only: there is one counting clock and no divider behind this

Bits	Name	Access	Reset	Description
				bit, so it reads back what was written and changes nothing
[0]	PIT_ENABLE	RW	0	Global PIT enable0 = PIT disabled (counters paused)1 = PIT enabled (counters active)

Programming Notes: - Setting `PIT_ENABLE=0` stops all counters immediately - Counters preserve their current count values when disabled - Recommended to disable PIT before reprogramming counters

---

11.3 PIT_CONTROL (0x004) - Control Word

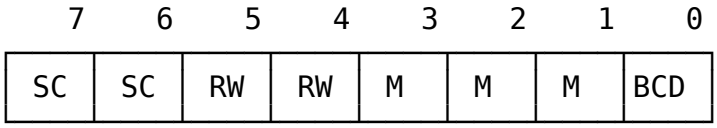
Access: Write-Only **Reset Value:** N/A

Bits	Name	Description
[7:6]	SC[1:0]	Counter Select`00` = Counter 0`01` = Counter 1`10` = Counter 2`11` = Read-back command (not implemented)
[5:4]	RW[1:0]	Read/Write Mode`00` = Counter latch command (see the latch note below)`01` = LSB only, bits [7:0]`10` = MSB only, bits [15:8]`11` = LSB then MSB, full 16-bit word (recommended)
[3:1]	M[2:0]	Counter Mode`000` = Mode 0 (Interrupt on terminal count)`001-101` = Modes 1-5 (not implemented)
[0]	BCD	Counting Mode`0` = Binary (16-bit, 0-65535)1 = BCD (4 digits, 0-9999)

Counter latch (RW=00) is the 8254 command. A control word with RW=00 freezes the selected counter's current count while the counter keeps running; it does not touch the counter's mode, RW mode or BCD programming. The next read of that counter's COUNTERx_DATA returns the frozen value and releases the latch, so reads after that see the live count again. A second latch command before the read is ignored – the first latched value survives. The latch is also released, without a read, when the counter is reprogrammed (a control word that programs it) or reloaded

(a data write), so the first read after a new program returns the new program's live count, never a stale snapshot of the old one. A data write never latches: RW=00 is the latch opcode, not a read/write mode, so a counter still at its reset RW=00 takes a data write as a full 16-bit load. (This replaced the write-triggered, sticky latch of issue #52 on 2026-09-09.)

Control Word Format (8254-Compatible):



Programming Example:

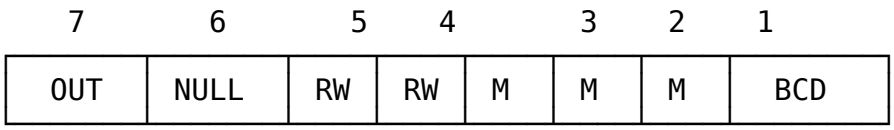
```
// Configure Counter 0 for Mode 0, binary, LSB+MSB access
uint32_t control_word = (0 << 6) |  // Counter 0
                        (3 << 4) |  // LSB+MSB
                        (0 << 1) |  // Mode 0
                        (0 << 0);   // Binary
write_register(PIT_CONTROL, control_word);  // Write 0x30
```

11.4 PIT_STATUS (0x008) - Status Readback

Access: Read-Only **Reset Value:** `0x00404040` (all counters in reset state: OUT=0, NULL_COUNT=1, RW_MODE/MODE/BCD=0 -> 0x40 per status byte)

Bits	Name	Description
[31:24]	RESERVED	Reserved, read as 0
[23:16]	COUNTER2_STATUS	Counter 2 status byte
[15:8]	COUNTER1_STATUS	Counter 1 status byte
[7:0]	COUNTER0_STATUS	Counter 0 status byte

Status Byte Format (per counter):



Bit	Name	Description
[7]	OUT	Counter OUT pin state0 = OUT low (counting)1 = OUT high (terminal count reached)

Bit	Name	Description
[6]	NULL_COUNT	No count loaded flag0 = Count value loaded1 = No count loaded yet
[5:4]	RW_MODE	Read/Write mode (mirrors control word)
[3:1]	MODE	Counter mode (mirrors control word). WARNING: this mirrors whatever mode was WRITTEN, but the counter logic implements Mode 0 only - firmware that programs Mode 2 reads back “Mode 2” while getting Mode 0 behavior.
[0]	BCD	BCD/Binary mode (mirrors control word)

Reading Example:

```
uint32_t status = read_register(PIT_STATUS);
uint8_t counter0_status = status & 0xFF;
bool out_high = (counter0_status >> 7) & 0x1;
bool null_count = (counter0_status >> 6) & 0x1;
uint8_t rw_mode = (counter0_status >> 4) & 0x3;
uint8_t mode = (counter0_status >> 1) & 0x7;
bool bcd = counter0_status & 0x1;
```

11.5 COUNTERx_DATA (0x010, 0x014, 0x018) - Counter Values

Access: Read/Write **Reset Value:** 0x00000000

Bits	Name	Access	Description
[31:16]	RESERVED	RO	Reserved, read as 0
[15:0]	COUNT	RW	Counter value (16-bit)

Write Behavior: - Program the control word first so the RW mode is what you intend. A counter that has never seen a control word (RW=00 out of reset) takes a data write as a full 16-bit load – it never latches - The byte the counter loads follows the RW mode, on its natural lane: - RW=11 (or reset RW=00): all 16 bits, count = PWDATA[15:0] - RW=01 (LSB only): count = {8'h00, PWDATA[7:0]} - RW=10 (MSB only): count = {PWDATA[15:8], 8'h00} – write 0xAB00 to load 0xAB00 - PSTRB is honoured: the register block merges the strobed bytes into the stored value and the load takes that merged value, so a PSTRB=0x1 write loads {stored high byte, new low byte} - The stored value that a byte-strobed write merges with is the field’s mirror of the LIVE count, not

the value software last wrote, so a partial load on a running counter takes the other byte from wherever the count is that cycle. Disable the PIT or use a full 16-bit write when the result must be deterministic. - One write, one load: the load strobe fires once, aligned to the stored value, so the count goes old to new in a single cycle with no stale intermediate value and no spurious OUT or interrupt. A running counter can be reloaded safely - A count of 0 means 65536 (10000 in BCD): terminal count after a full wrap - The load clears NULL_COUNT and drives OUT low; the counter then counts whenever GATE is high and PIT_ENABLE=1. GATE low pauses it, GATE high resumes it from where it stopped - Writes while counting replace the count and restart from the new value

Read Behavior: - The value returned depends on the counter's programmed RW mode, on the same lane the write uses: - RW=01 (LSB only): returns {8'h00, count[7:0]} - RW=10 (MSB only): returns {count[15:8], 8'h00} – the high byte in bits [15:8], bits [7:0] read 0 - RW=11 (LSB then MSB): returns the full 16-bit current count - Returns the live count – or, if a latch command is pending, the latched count; that read releases the latch (see PIT_CONTROL) - After terminal count the counter parks at 0 with OUT high; it does not keep decrementing past zero the way a real 8254 does (stated deviation) - The counter continues decrementing while being read; for an atomic snapshot use the latch command, or disable the PIT first

Programming Example:

```
// Program Counter 0 with count value 1000
write_register(PIT_CONTROL, 0x30);  // Counter 0, LSB+MSB, Mode 0
write_register(COUNTER0_DATA, 1000);

// Counter will:
// 1. Load 1000 into internal counter
// 2. Start decrementing if GATE=1 and PIT_ENABLE=1
// 3. Set OUT=1 when count reaches 0
```

Read Example:

```
// Read current count value
uint32_t count = read_register(COUNTER0_DATA) & 0xFFFF;
```

11.6 Timing

Write Timing:

```
APB Write → field storage updates (PSTRB merged) → one aligned load
strobe,
next cycle → counter holds the new value → counting while GATE high
and PIT enabled
```

Read Timing:

```
APB Read → Counter Sample (1 cycle) → Register Read (1 cycle) → APB
Response
```

Important: Due to APB and counter pipeline delays, there may be 2-3 cycle latency between register writes and counter response.

11.7 Usage Example

Basic Counter Start:

```
// 1. Disable PIT
write_register(PIT_CONFIG, 0x00);

// 2. Program control word
write_register(PIT_CONTROL, 0x30);  // Counter 0, Mode 0, binary

// 3. Load count value
write_register(COUNTER0_DATA, 1000);

// 4. Enable PIT
write_register(PIT_CONFIG, 0x01);

// 5. Wait for OUT signal or poll status
while (!(read_register(PIT_STATUS) & 0x80)) {
    // Wait for OUT bit to go high
}
```

Multiple Counter Configuration:

```
// Disable PIT
write_register(PIT_CONFIG, 0x00);

// Program Counter 0
write_register(PIT_CONTROL, 0x30);
write_register(COUNTER0_DATA, 100);

// Program Counter 1
write_register(PIT_CONTROL, 0x70);  // Counter 1
write_register(COUNTER1_DATA, 200);

// Program Counter 2
write_register(PIT_CONTROL, 0xB0);  // Counter 2
write_register(COUNTER2_DATA, 300);

// Enable all counters
write_register(PIT_CONFIG, 0x01);
```

Version: 1.1 **Last Updated:** 2026-09-09

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

12 8254 PIT (Programmable Interval Timer) - APB Implementation

Status: Mode 0 complete and defect-clean against GitHub #52 **Priority:** High **Address:** `0xFEC0_2000 - 0xFEC0_2FFF` (4KB window)

Test status: 31/32 in all six configurations (CDC_ENABLE 0/1 x gate/func/full, 2026-09-09). The one failure, `test_gh52_latch_cleared_by_reprogram`, is a measurement taken through an APB read whose own round trip exceeds the window it asserts - see "Measuring a counter through the bus" below. `test_gh52_gate_pause_resume` had the same problem in its first draft and was resolved by sampling the counter white-box around the GATE edge.

12.1 Overview

APB-based implementation of 8254-compatible Programmable Interval Timer with 3 independent 16-bit counters.

Follows HPET 3-layer architecture pattern with PeakRDL-generated registers.

12.2 Features

- 8254-compatible register interface
- 3 independent 16-bit counters
- Mode 0: interrupt on terminal count, with GATE pause/resume
- Modes 1-5: NOT implemented - the mode field is stored and reported by read-back, but every mode counts like mode 0
- Binary and BCD counting; a count of 0 means 65536 (BCD: 10000)
- Counter latch COMMAND (control word RW = 00): freezes the count, the next data read returns it and releases the latch
- LSB / MSB / 16-bit load lanes, byte-strobe (PSTRB) correct
- Strict address decode: only the seven mapped registers are visible, every other address in the 4 KB window is dropped with PSLVERR
- Interrupt output array: `timer_irq[2:0]`
- APB4 slave interface
- Optional CDC (clock domain crossing)
- `gate_in` input synchronizer (SYNC_STAGES, default 2), unconditional

12.2.1 GATE (mode 0)

GATE	Effect
low	counting SUSPENDS; the count holds its value
high	counting RESUMES from that value - no reload, no restart
any	a counter LOAD is unaffected; only the decrement is gated

12.2.2 Counting-block priority

```
control word  >  counter load  >  count tick
```

A control word ABORTS the count it targets - OUT low, NULL COUNT set, and no tick in that cycle, so the count freezes at the value the abort caught. A load starts a new count. Only when neither happened does the counter decrement. These are one if/else-if chain on purpose: as separate statements a control word landing on the terminal tick had its OUT clear overwritten by the tick, raising a spurious interrupt with NULL COUNT set.

12.2.3 Counter latch

A control word with RW = 00 is the latch COMMAND, not a read/write mode. It freezes the selected counter's current count while the counter keeps running. A second latch before the read is ignored (the first snapshot survives), and a DATA write never latches - out of reset (RW = 00) a data write is a 16-bit load.

The latch is held until one of:

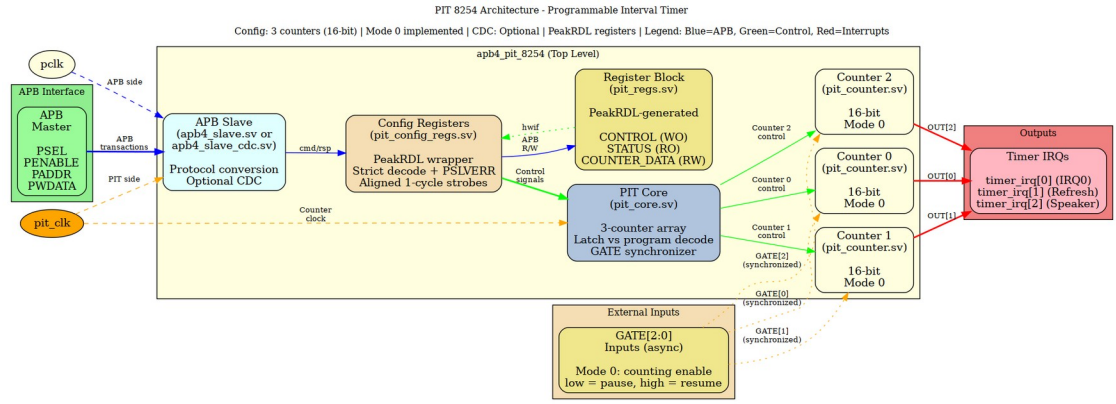
- the counter's data register is READ - that read returns the frozen value and releases the latch, and reads after it see the live count again;
- the counter is REPROGRAMMED by a control word;
- a new count is LOADED.

The last two matter: the snapshot describes a count that no longer exists after a reprogram, and a latch that survived one would hand software a stale value on its next read, indistinguishable from a live count.

12.2.4 Byte-strobed loads

COUNTERx_DATA is `hw = rw` and its hardware input is the live count, so the register field MIRRORS the counter rather than shadowing the last value written. A byte-strobed (partial PSTRB) write merges the new byte with the counter's value as of the write cycle. A partial load is therefore deterministic only when the counter is stopped (PIT disabled); a full 16-bit write is deterministic always, because no lane comes from the mirror.

12.3 Architecture



PIT 8254 Architecture

(Diagram source: `docs/assets/graphviz/pit_architecture.gv`; render the PNG with the Makefile in that directory if it is not present.)

Figure 1: PIT 8254 module architecture showing APB interface, PeakRDL-generated registers, and 3 independent counter channels with Mode 0 implementation.

12.3.1 Module Hierarchy

```
apb4_pit_8254.sv          → APB wrapper, CDC select
  ├── pit_config_regs.sv → Strict decode + PSLVERR, aligned command
strobes
  │     └── pit_regs.sv  → PeakRDL generated registers
  └── pit_core.sv         → 3-counter array, control decode, GATE
synchronizer
        └── pit_counter.sv → Single counter (mode 0)
```

12.3.2 Parameters

Parameter	Default	Meaning
`NUM_COUNTERS`	3	Fixed at 3 - the register map is generated for three counters
`CDC_ENABLE`	0	1 = registers and counters run on `pit_clk`, the cmd/rsp stream crosses in `apb4_slave_cdc`
`USE_JOHNSON`	0	Async-FIFO pointer encoding for the CDC

Parameter	Default	Meaning
		block
SYNC_STAGES	2	`gate_in` synchronizer depth, >= 2

12.4 Register Map

Seven 32-bit registers at word offsets 0x000-0x018. The authoritative map lives in the MAS register chapter: ch05_registers/01_register_map.md.

Nothing else in the 4 KB window is decoded. Any other address is dropped - write ignored, read returns zero - and answers with PSLVERR.

Counter data lanes follow the control word's RW field:

RW	Write	Read
00	16-bit load (latch is the CONTROL-word command, not a data write)	16-bit
01	low byte from PWDATA[7:0]	`{8'h00, count[7:0]}`
10	high byte from PWDATA[15:8]	`{count[15:8], 8'h00}` - the same lane both ways; low byte reads 0
11	16-bit load	16-bit

12.5 Counter Modes

Mode	Name	Status
0	Interrupt on terminal count	Complete
1	Hardware retriggerable one-shot	TODO
2	Rate generator	TODO
3	Square wave generator	TODO
4	Software triggered strobe	TODO

Mode	Name	Status
5	Hardware triggered strobe	TODO

12.6 Interrupt Outputs

Following HPET pattern: - `timer_irq[0]` = Counter 0 OUT (system timer, IRQ0) - `timer_irq[1]` = Counter 1 OUT (DRAM refresh or general) - `timer_irq[2]` = Counter 2 OUT (PC speaker or general)

12.7 Example Instantiation

```
apb4_pit_8254 #(
    .NUM_COUNTERS (3),
    .CDC_ENABLE   (0),
    .SYNC_STAGES  (2)
) u_pit (
    .pclk          (apb_clk),
    .presetn       (apb_rst_n),
    .pit_clk       (timer_clk),
    .pit_resetn    (timer_rst_n),
    .s_apb_PSEL    (psel_pit),
    .s_apb_PENABLE (penable),
    .s_apb_PREADY  (pready_pit),
    .s_apb_PADDR   (paddr),
    .s_apb_PWRITE  (pwrite),
    .s_apb_PWDATA  (pwdata),
    .s_apb_PSTRB   (pstrb),
    .s_apb_PPROT   (pprot),
    .s_apb_PRDATA  (prdata_pit),
    .s_apb_PSLVERR (pslverr_pit),
    .gate_in       (gate[2:0]),
    .timer_irq     (pit_irq[2:0])
);
```

(The port list above is the real one. The example here used to name `paddr`, `psel`, `pit_rst` and friends, none of which exist on this module.)

12.8 Files

- `apb4_pit_8254.sv` - Top-level APB wrapper
- `pit_config_regs.sv` - Register wrapper with edge detection
- `pit_core.sv` - 3-counter array
- `pit_counter.sv` - Single counter (mode 0)
- `pit_regs.sv` - PeakRDL generated registers

- `pit_regs_pkg.sv` - PeakRDL generated package
- `../../rdl/pit_8254/pit_regs.rdl` - SystemRDL specification

12.9 Development Status

- ☒ SystemRDL register specification
- ☒ PeakRDL register generation
- ☒ Counter mode 0 logic implementation
- ☒ Core PIT logic (3-counter array)
- ☒ APB wrapper
- ☒ Basic testbench (gate level, 6 tests)
- ☒ Medium testbench (GitHub #52 defect regression, 13 tests)
- ☒ Full testbench (13 further tests)
- ☐ Modes 1-5 implementation
- ☐ Read-back command support (control word SC = 11)

12.10 Stated deviations from the 8254

1. **Modes 1-5 do not exist.** The mode field is stored and reported; the counting logic has no `case (cfg_mode)`.
2. **The counter parks at terminal count.** A real 8254 keeps decrementing past 0 and wraps with OUT high. Here `r_counting` clears, the count stays at 0 and OUT stays high until a new load. This is the fix for the post-terminal `r_counting` oscillation in #52; the steady state is `r_counting = 0, r_count = 0, r_out = 1`.
3. **RW = 10 uses bits [15:8] in both directions.** The 8254 presents the selected byte on its 8-bit bus; on this 16-bit register the high byte is written and read on the high lane, so a write of 0xAB00 reads back 0xAB00.
4. **Read-back command (SC = 11) is not implemented** - PIT_STATUS carries the same information as a register instead.
5. **`PIT_CONFIG.clock_select` is storage with no hardware effect** - there is one counting clock and no prescaler to select between.

12.11 Compliance

- Reset macros (`ALWAYS_FF_RST`)
- HPET architecture pattern
- APB4 standard interface

- PeakRDL register generation

Last Updated: 2026-09-09 **Status:** Mode 0 complete; GitHub #52 RTL defects fixed